



**Kutaisi International University**  
**School of Computer Science**  
**Bachelor's Program in Computer Science**

**Kopaliani Mate, Kajaia Zura**

**Implementation of IEEE-754 Standard Compliant Floating  
Point Unit**

The thesis is completed for obtaining the academic degree of Bachelor  
of Computer Science

**Advisor:** Prof. Dr. Walter F. Tichy  
**Co-advisor:** Prof. Dr. Wolfgang J. Paul  
**Submission Date:** 30.06.2026

Kutaisi, 2026

# Declaration

We declare that this thesis has been composed solely by ourselves and that it has not been submitted, in whole or in part, in any previous application for a degree. Except where stated otherwise by reference or acknowledgment, the work presented is entirely our own.

**Note on Language Assistance:** An AI tool was used only to help improve the grammar and better wording of the thesis. All technical work presented in this thesis, including the design, architecture, SystemVerilog implementation and verification test suite, was completed entirely by authors.

Kutaisi, 30.06.2026

Kopaliani Mate, Kajaia Zura

## **Abstract**

This thesis presents the hardware implementation and verification of a Floating Point Unit (FPU) following the IEEE-754 standard. The goal of the project is to take the theoretical computer architecture blueprints from the Mueller and Paul textbook and translate them into functional hardware blocks using SystemVerilog. The implementation is divided into four core modules: an unpacker for data field extraction and special-case identification, unified addition and subtraction unit with operand alignment pipelines, unified combinational multiplier and iterative divider managed by custom finite state automaton utilizing Newton-Raphson estimation, and multi-mode compliant rounder. Functional hardware simulations were run alongside an automated testing framework written in Rust to compare thousands of randomized and edge-case inputs directly against software reference models.

**Keywords:** Floating-Point Unit, IEEE-754, SystemVerilog, Hardware Implementation

# Contents

|          |                                                   |           |
|----------|---------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction .....</b>                         | <b>1</b>  |
| <b>2</b> | <b>Floating Point Representation .....</b>        | <b>4</b>  |
| <b>3</b> | <b>Unpacker .....</b>                             | <b>8</b>  |
| 3.1      | Unpacking .....                                   | 9         |
| 3.2      | Special-Case Detection (SPECUNP) .....            | 12        |
| 3.3      | NaN Selection .....                               | 13        |
| 3.4      | Simulation .....                                  | 14        |
| <b>4</b> | <b>Addition and Subtraction Unit .....</b>        | <b>16</b> |
| 4.1      | Alignment Shift .....                             | 17        |
| 4.1.1    | EXPSUB .....                                      | 18        |
| 4.1.2    | LIMIT .....                                       | 19        |
| 4.1.2.1  | OR Tree. ....                                     | 21        |
| 4.1.3    | Swapping Operands .....                           | 22        |
| 4.1.4    | Sticky Bit Computation .....                      | 23        |
| 4.1.5    | Half Decoder .....                                | 25        |
| 4.2      | Significand Add .....                             | 27        |
| 4.3      | Special Cases .....                               | 30        |
| 4.4      | Sign Computation .....                            | 31        |
| <b>5</b> | <b>Multiplication and Division Unit .....</b>     | <b>33</b> |
| 5.1      | Sign and Exponent Computation .....               | 41        |
| 5.2      | Multiplication and Division of significands ..... | 46        |
| 5.2.1    | Quotient structure .....                          | 47        |
| 5.2.2    | Reciprocal approximation .....                    | 47        |
| 5.2.3    | Newton–Raphson Iteration .....                    | 49        |
| 5.2.3.1  | Finite-precision iteration .....                  | 49        |
| 5.2.3.2  | Simplified subtraction .....                      | 49        |
| 5.2.3.3  | Iteration form .....                              | 50        |
| 5.2.3.4  | Quotient reconstruction .....                     | 50        |
| 5.3      | The steps of significands division .....          | 50        |

|                                            |           |
|--------------------------------------------|-----------|
| 5.4 Exception Flags .....                  | 61        |
| <b>6 Rounder .....</b>                     | <b>63</b> |
| 6.1 Normalization Shift .....              | 64        |
| 6.2 Flags .....                            | 64        |
| 6.3 Exponent Normalization .....           | 69        |
| 6.4 Shift Distance .....                   | 71        |
| 6.5 Significand Normalization Shift .....  | 72        |
| 6.5.1 Cyclic left shifter .....            | 73        |
| 6.5.2 Mask .....                           | 74        |
| 6.6 Selection of the representatives ..... | 76        |
| 6.7 Significand Rounding .....             | 77        |
| 6.7.1 Rounding Decision .....              | 78        |
| 6.8 Post Normalization .....               | 79        |
| 6.9 Exponent Adjustment .....              | 80        |
| 6.10 Exponent Rounding .....               | 82        |
| 6.11 Circuit SPECFPRND .....               | 83        |
| 6.11.1 SPECSELECT .....                    | 84        |
| 6.11.2 RNDEXCEPTIONS .....                 | 85        |
| 6.12 Operation with Special Cases .....    | 86        |
| 6.12.1 NAN signal .....                    | 86        |
| 6.12.2 Addition/Subtraction .....          | 87        |
| 6.12.3 Multiplication .....                | 88        |
| 6.12.4 Division .....                      | 88        |
| <b>7 Review And Result Analysis .....</b>  | <b>89</b> |
| 7.1 Adder .....                            | 89        |
| 7.2 Subtractor .....                       | 91        |
| 7.3 Multiplier .....                       | 92        |
| 7.4 Divider .....                          | 93        |
| 7.5 Design .....                           | 94        |
| 7.6 Test Case Generation .....             | 95        |
| 7.7 Results .....                          | 96        |
| <b>8 Conclusion .....</b>                  | <b>98</b> |
| <b>Bibliography .....</b>                  | <b>99</b> |

# 1 Introduction

This thesis presents the SystemVerilog implementation of a gate-level floating-point unit (**FPU**) based on the IEEE-754 standard. The purpose of the project is to demonstrate how floating-point arithmetic can be constructed from hardware modules rather than treated only as a software-level abstraction. Thesis contributes a translation of the design from the textbook by Mueller and Paul [1] into a working SystemVerilog system. The implementation follows the textbook structure where possible, while the necessary modifications and implementation choices are explained in the relevant sections of the text.

Floating-point arithmetic is a fundamental component of modern computer systems, but correctness is highly nontrivial as discussed in [1], ensuring correctness in computer arithmetic requires careful handling of both numerical representation and hardware-level details. For this reason, the correctness considerations in this thesis are explained primarily in terms of the textbook design.

IEEE-754 floating-point numbers are stored in a packed binary format. A value is represented using a sign, an exponent, and a significand. In this thesis, this representation is written as

$$(s, e, f) \tag{1}$$

where  $s$  is the sign bit,  $e$  is the exponent field, and  $f$  is the fraction/significand field. Before arithmetic can be performed, these fields must be extracted and interpreted. After the operation is complete, the result must be normalized, rounded, and packed back into IEEE-754 format. The implementation is divided into 4 main components: an unpacker, an adder/subtractor, a multiplier/divider, and a rounder. These components form the main datapath of the FPU, starting from packed input operands and ending with a packed floating-point result. We divided this components as following: Mate Kopaliani implemented the unpacker, the multiplier/divider, and the Rust-based software testing framework. Zura Kajaia implemented the adder/subtractor and the rounder.

- **Unpacker** is the first stage of the design. It extracts the sign, exponent, and significand fields from the input operands and detects special cases such as zero, infinity, and NaN. This stage prepares the operands  $fa, fb$  for the arithmetic units and prevents special values from being handled as ordinary numbers.

- **Adder/Subtractor** performs floating-point addition and subtraction in a shared datapath. Since floating-point operands may have different exponents, the significands must first be aligned before the operation is performed. Subtraction is handled through sign logic, allowing both addition and subtraction to use the same core structure.
- **Multiplier** computes the product of two floating-point operands by handling the sign, exponent, and significand separately. The result sign is produced from the operand signs, the exponents are combined, and the significands are multiplied. The intermediate result is then normalized and passed to the rounding stage.
- **Divider** uses Newton-Raphson iteration to compute division. Unlike addition and multiplication, division is iterative and does not complete in a single fixed combinational path. Because of this, the divider requires separate control logic. In this design, that control logic is handled by a division automaton, which manages the iterative steps.
- **Rounder** is the final stage of the FPU. It receives the intermediate result from the arithmetic units and applies the selected rounding mode. The rounder also handles cases where rounding changes the normalized form of the result, requiring exponent correction before the final value is packed.

The complete system was implemented in SystemVerilog. Each component was developed as a separate module and then connected into the full FPU. This modular structure makes it easier to test individual parts of the design and to describe the role of each stage in the datapath.

Testing was performed using SystemVerilog simulations and the Rust-based testing framework. Rust was used to generate reference values and compare them with the hardware outputs. This helped verify normal arithmetic cases as well as edge cases involving special operands, rounding, and exception flags. The result of this thesis is a SystemVerilog realization of the textbook gate-level IEEE-754 floating-point unit, with separate modules for unpacking, addition/subtraction, multiplication, division, and rounding. The following chapters describe the floating-point format, the internal structure of the FPU, the implementation of each component, the modifications made during translation, and the verification process.

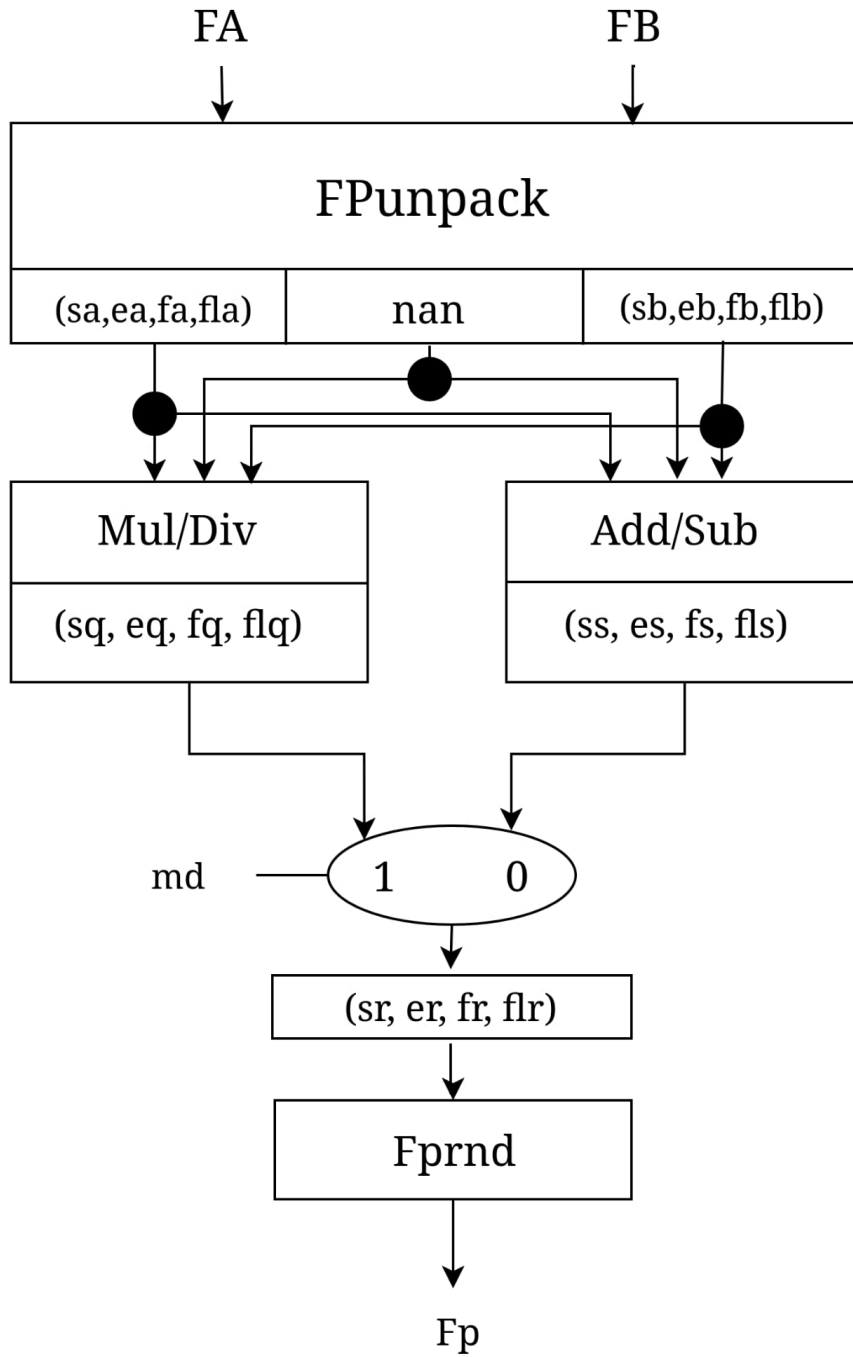


Figure 1: Overview of the design



## 2 Floating Point Representation

Before introducing the unpacker unit, it is useful to define the floating-point format used throughout the FPU. The unpacker, arithmetic units, and rounder all depend on the same interpretation of the sign, exponent, and significand fields.

An IEEE-754 floating-point number is stored in a packed format consisting of three fields: the sign bit, the exponent field, and the fraction field. In this thesis, the fields are written as

$$s \in \mathbb{B}, \quad e[n-1:0] \in \mathbb{B}^n, \quad f[1:p-1] \in \mathbb{B}^{p-1} \quad (2)$$

where  $n$  is the number of exponent bits and  $p$  is the precision of the significand, including the implicit leading bit.

For single and double precision, these parameters are

$$(n, p) = \begin{cases} (8, 24) & \text{for single precision} \\ (11, 53) & \text{for double precision} \end{cases} \quad (3)$$

The sign bit determines whether the number is positive or negative. The exponent controls the scale of the number, while the fraction field stores the remaining bits of the significand. For normal numbers, the leading significand bit is implicit, so the stored fraction  $f$  represents the significand  $\langle 1.f \rangle$ . For denormal numbers, the leading bit is not implicit, so the significand is interpreted as  $\langle 0.f \rangle$ .

The exponent field is stored as a biased integer. Instead of storing the exponent as a signed value, IEEE-754 stores it as an unsigned value with a fixed bias. For an  $n$ -bit exponent field, the bias is

$$bias_n = 2^{n-1} - 1 \quad (4)$$

If  $\langle e \rangle$  is the unsigned value represented by the exponent field, then the actual exponent for normal numbers is

$$[[e]]_{bias} = \langle e \rangle - bias_n \quad e \notin \{0^n, 1^n\} \quad (5)$$

The all-zero and all-one exponent values are not used for normal numbers. Therefore, the minimum and maximum normal exponents are

$$\begin{aligned}
e_{min} &= 1 - bias_n = 1 - (2^{n-1} - 1) = -2^{n-1} + 2 \\
e_{max} &= (2^n - 2) - bias_n = (2^n - 2) - (2^{n-1} - 1) = 2^{n-1} - 1
\end{aligned} \tag{6}$$

With these definitions, the value represented by a floating-point tuple  $(s, e, f)$  is

$$[[s, e, f]] = \begin{cases} (-1)^s \cdot 2^{[[e]]_{bias}} \cdot \langle 1.f \rangle & \text{if } e \notin \{0^n, 1^n\} \\
(-1)^s \cdot 2^{e_{min}} \cdot \langle 0.f \rangle & \text{if } e = 0^n \\
(-1)^s \cdot \infty & \text{if } e = 1^n \text{ and } f = 0^{p-1} \\
NaN & \text{if } e = 1^n \text{ and } f \neq 0^{p-1} \end{cases} \tag{7}$$

According to this encoding, an IEEE-754 floating-point number is classified as

- **normal** when  $e \notin \{0^n, 1^n\}$
- **zero** when  $e = 0^n$  and  $f = 0^{p-1}$
- **denormal** when  $e = 0^n$  and  $f \neq 0^{p-1}$
- **infinity** when  $e = 1^n$  and  $f = 0^{p-1}$
- **NaN** when  $e = 1^n$  and  $f \neq 0^{p-1}$

The arithmetic units do not operate directly on biased exponents. After the unpacker classifies the operands, normal exponent values are converted into two's complement form. Arithmetic is then performed on the two's complement exponents, and the final exponent is converted back into biased form before packing the result.

For an  $n$ -bit two's complement number  $y[n-1:0]$ , the represented value is

$$[y[n-1:0]] = -y_{n-1} \cdot 2^{n-1} + \langle y[n-2:0] \rangle \tag{8}$$

then  $y$  must be in the following range

$$[y[n-1:0]] \in \{-2^{n-1}, \dots, 2^{n-1} - 1\} \tag{9}$$

Let  $x[n-1:0]$  be a biased exponent field and  $y[n-1:0]$  be the corresponding two's complement exponent. For normal exponent values, the conversion must satisfy

$$[[x]]_{bias} = [y] \tag{10}$$

Using the definition of the biased exponent, we get

$$\begin{aligned}
\leftrightarrow \langle x \rangle - 2^{n-1} + 1 &= -y_{n-1} \cdot 2^{n-1} + \langle y[n-2:0] \rangle \\
\leftrightarrow \langle x \rangle + 1 &= 2^{n-1} \cdot (1 - y_{n-1}) + \langle y[n-2:0] \rangle \\
&= \langle \overline{y_{n-1}} \rangle y[n-2:0]
\end{aligned} \tag{11}$$

This gives a simple conversion from biased representation to two's complement:

1. interpret  $x$  as an unsigned binary number and add 1

2. invert the most significant bit of the result

The inverse conversion is used when the final exponent must be packed back into IEEE-754 format. Solving the same equation for  $x$  gives

$$\langle x \rangle = [y] + 2^{n-1} - 1 = \langle y \rangle + 2^{n-1} - 1 \pmod{2^n} \quad (12)$$

Equivalently, converting from two's complement back to biased representation can be done by inverting the most significant bit and subtracting 1.

This conversion is only applied to exponent fields that represent normal exponents. The special exponent patterns  $0^n$  and  $1^n$  are handled separately by the unpacker because they represent zero, infinity, and NaN values.

We also need the notion of an  $\alpha$ -representative before defining the rounding functions. For a fixed integer  $\alpha$ , the real line is partitioned by the endpoints  $q \cdot 2^{-\alpha}$ , where  $q \in \mathbb{Z}$ . Two real numbers  $x$  and  $y$  are called  $\alpha$ -equivalent if they are in the same equivalence class of this partition: they either lie in the same open interval or both coincide with the same endpoint. We write this as  $x \underset{\alpha}{=} y$ . Thus, for some integer  $q$ ,

$$\begin{aligned} x \underset{\alpha}{=} y &\Leftrightarrow x, y \in (q \cdot 2^{-\alpha}, (q+1) \cdot 2^{-\alpha}) \\ \text{or } x &= y = q \cdot 2^{-\alpha} \end{aligned} \quad (13)$$

From each equivalence class, we choose a representative. For singleton classes there is no choice, and from each open interval the midpoint is selected. This defines the  $\alpha$ -representative of a real number  $x$  as

$$[x]_{\alpha} = \begin{cases} (q + 0.5) \cdot 2^{-\alpha} & \text{if } x \in (q \cdot 2^{-\alpha}, (q+1) \cdot 2^{-\alpha}) \\ x & \text{if } x = q \cdot 2^{-\alpha} \end{cases} \quad (14)$$

for some integer  $q$ .

While the encoding determines how floating-point numbers are represented internally, it does not guarantee that the result of an arithmetic operation can be expressed exactly in this format. In general, the exact result may fall outside the set of representable numbers, making an additional approximation step necessary.

To formalize this, let  $\mathcal{R}$  denote the set of representable floating-point numbers, and define

$$\mathcal{R}_{\infty} = \mathcal{R} \cup \{\infty, -\infty\}, \quad (15)$$

since  $\mathcal{R}$  is not closed under arithmetic operations. The rounding stage therefore implements a function

$$r : \mathbb{R} \rightarrow \mathcal{R}_\infty \quad (16)$$

which maps a real value  $x$  to the nearest representable result according to a selected rounding mode. The IEEE standard defines four rounding modes:

- $r_u$  round up
- $r_d$  round down
- $r_z$  round to zero
- $r_{ne}$  round to nearest even

The behavior of rounding near the limits of the representable range depends on the extremal floating-point values. The smallest and largest positive normal representable numbers are given by

$$X_{\min} = 2^{e_{\min}} \cdot 2^{-(p-1)} \quad (17)$$

$$X_{\max} = 2^{e_{\max}} (2 - 2^{-(p-1)}) \quad (18)$$

Values smaller than  $X_{\min}$  fall into the subnormal range, while values exceeding  $X_{\max}$  cannot be represented as finite numbers. These bounds will be used in the precise definition of rounding behavior.

The first three rounding modes have straightforward interpretations. For a real value  $x$ , they are defined as

$$\begin{aligned} r_u(x) &= \min\{y \in \mathcal{R}_\infty \mid x \leq y\} \\ r_d(x) &= \max\{y \in \mathcal{R}_\infty \mid y \leq x\} \\ r_z(x) &= \begin{cases} r_d(x) & \text{if } x \geq 0 \\ r_u(x) & \text{if } x \leq 0 \end{cases} \end{aligned} \quad (19)$$

The fourth rounding mode, round to nearest, ties to even, requires a more careful definition. For any  $x$  such that  $-X_{\max} \leq x \leq X_{\max}$ , the value  $r_{ne}(x)$  is defined as the representable number  $y$  closest to  $x$ . If two such values exist (i.e.,  $x$  lies exactly halfway between two representable numbers), the one with an even significand is chosen. Let

$$X_{\max}^* = 2^{e_{\max}} (2 - 2^{-n}) \quad (20)$$

denote the largest finite value whose successor would overflow. Then, for values outside the normal range, the rounding rule is extended as

$$r_{ne}(x) = \begin{cases} \infty & \text{if } x \geq X_{\max}^* \\ X_{\max} & \text{if } X_{\max} < x < X_{\max}^* \\ -X_{\max} & \text{if } -X_{\max}^* < x < -X_{\max} \\ -\infty & \text{if } x \leq -X_{\max}^* \end{cases} \quad (21)$$

In this design, the selected rounding mode is encoded using the two-bit field  $RM[1 : 0]$ :

| RM[1 : 0] | symbol   | rounding modes        |
|-----------|----------|-----------------------|
| 00        | $r_z$    | round to zero         |
| 01        | $r_{ne}$ | round to nearest even |
| 10        | $r_u$    | round up              |
| 11        | $r_d$    | round down            |

### 3 Unpacker

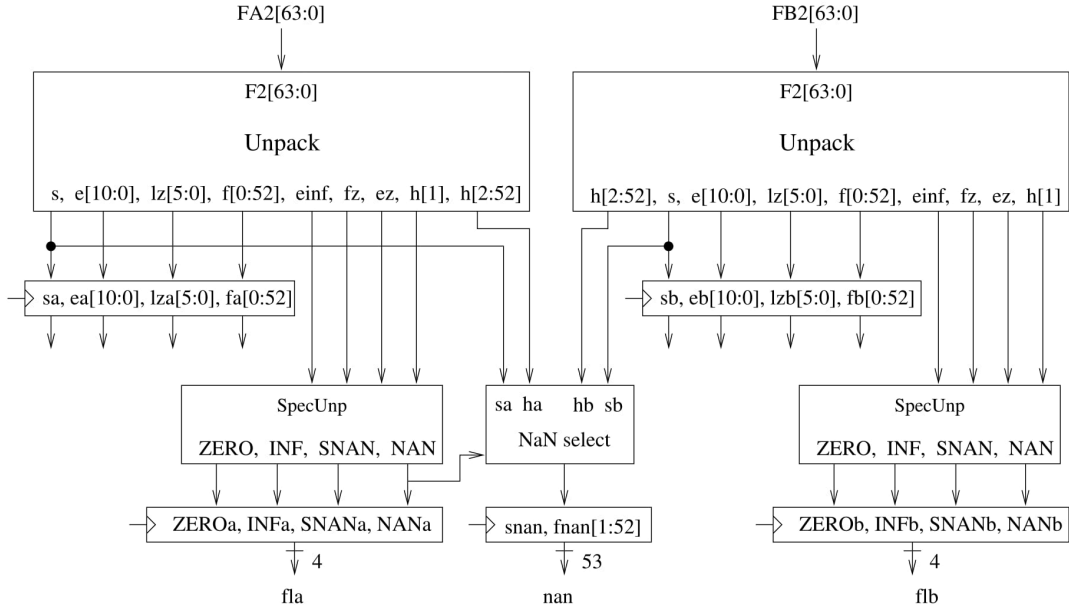


Figure 2: Overview of the unpacker design

The unpacker is the first stage of the floating-point unit. Its role is to convert the packed IEEE-754 input operand into an internal representation used by the arithmetic units. This includes extracting the sign bit, converting the exponent, constructing the significand, and detecting special operand classes.

The top-level unpacker module is divided into three main parts: **UNPACK**, **SPECUNP**, and **NANSELECT**. The **UNPACK** circuit extracts the operand fields and prepares the internal exponent and significand. The **SPECUNP** circuit detects special cases such as zero, infinity, quiet NaN, and signaling NaN. The **NANSELECT** circuit selects the NaN payload used when an invalid operation must produce a quiet NaN.

### 3.1 Unpacking

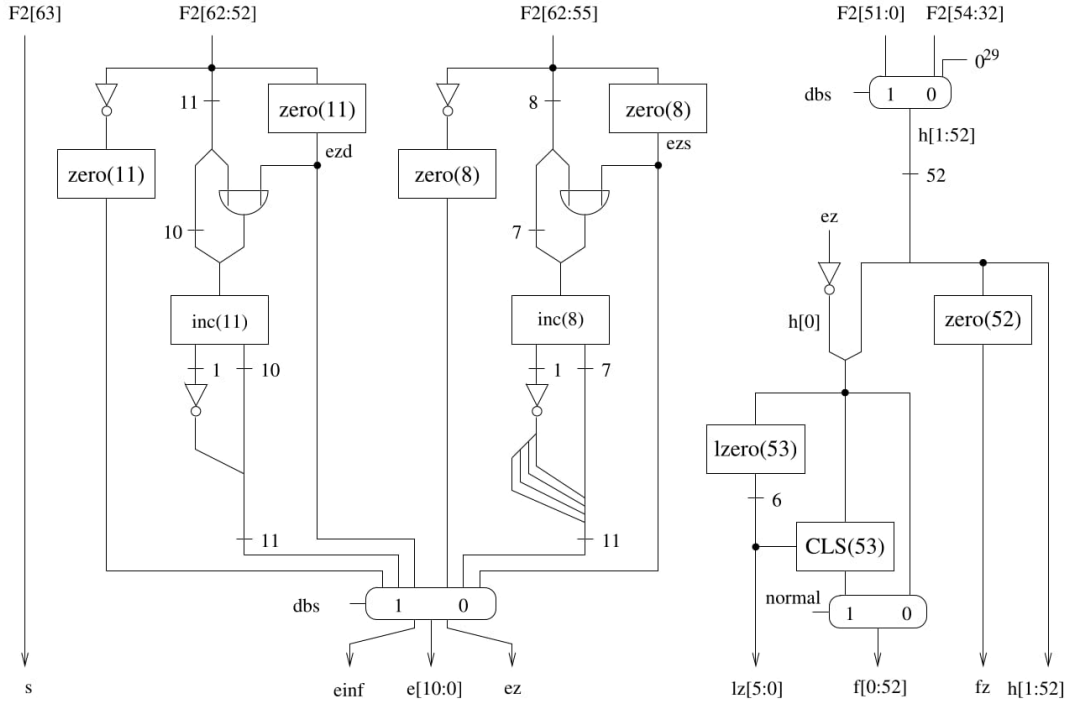


Figure 3: Design of the UNPACK circuit

The unpacking unit receives a 64-bit input operand  $F_A \in \mathbb{B}^{64}$ , a precision selection signal  $db \in \mathbb{B}$ , and a normalization control signal  $normal \in \mathbb{B}$ . When  $db = 1$ , the input is interpreted as a double-precision operand. When  $db = 0$ , the input is interpreted as a single-precision operand stored in the upper part of the 64-bit input.

Formally, the input fields are selected as

$$(s, e_{in}[n-1:0], f_{in}[1:p-1]) = \begin{cases} (F_A[63], F_A[62:52], F_A[51:0]) & \text{if } db = 1 \\ (F_A[63], F_A[62:55], F_A[54:32]) & \text{if } db = 0 \end{cases} \quad (22)$$

The unpacking logic is split into exponent and significand processing. The exponent logic detects whether the exponent field is all zeros or all ones and converts normal biased exponent values into two's complement form (Equation 11). The signals *ez* and *eof* are generated during this stage. The signal *ez* is true when the exponent field is zero, and *eof* is true when the exponent field contains only ones.

```

1  module exponent (
2      input [63:0]    fp,
3      input          db,
4      output          e_eof,
5      output          e_z,
6      output [10:0]   e
7  );
8
9  wire [10:0] temp_e_db;
10 wire [7:0] temp_e_sp;
11 wire [10:0] inc_db;
12 wire [7:0] inc_sp;
13 assign e_zd = (fp[62:52] == 11'b0);
14 assign e_zs = (fp[62:55] == 8'b0);
15 assign e_z = db ? e_zd : e_zs;
16 assign e_eof = db ? (~fp[62:52] == 11'b0) : (~fp[62:55] == 8'b0);
17 assign temp_e_db = {fp[62:53], e_zd | fp[52]};
18 assign temp_e_sp = {fp[62:56], e_zs | fp[55]};
19 assign inc_db = temp_e_db + 1;
20 assign inc_sp = temp_e_sp + 1;
21 assign e = db ? {~inc_db[10], inc_db[9:0]} : { {4{~inc_sp[7]}}, inc_sp[6:0] };
22 endmodule

```

In the significand part for normal numbers the hidden leading bit is inserted. For zero and denormal numbers, the leading bit is cleared. In double precision, the 52-bit fraction is used directly. In single precision, the 23-bit fraction is placed into the upper part of the internal 52-bit fraction field and the remaining bits are filled with zeros.

```

1  module significant (
2      input          db,
3      input [63:0]   fp,
4      input          e_z,
5      input          normal,

```

```

6     output [5:0] lz,
7     output [52:0] f,
8     output      fz,
9     output [52:0] h
10 );
11
12 parameter n = 64;
13 parameter m = 6;
14
15 wire [63:0] temp;
16 wire [52:0] te;
17 wire [52:0] temp1;
18 wire [6:0] lz_out;
19
20 leadingzero #(n,m) leadingzero_inst (
21     .x(temp),
22     .y(lz_out)
23 );
24
25 cls CLS_inst (
26     .m(lz),
27     .x(te),
28     .y(temp1)
29 );
30
31 assign h[52] = ~e_z;
32 assign h[51:0] = db ? fp[51:0] : {fp[54:32], 29'b0};
33 assign temp = {~e_z, h[51:0], {11{1'b1}}};
34 assign te = {~e_z, h[51:0]};
35 assign fz = (52'b0 == h[51:0]);
36 assign f = normal ? temp1 : te;
37 assign lz = lz_out[5:0];
38 endmodule

```



### 3.2 Special-Case Detection (SPECUNP)

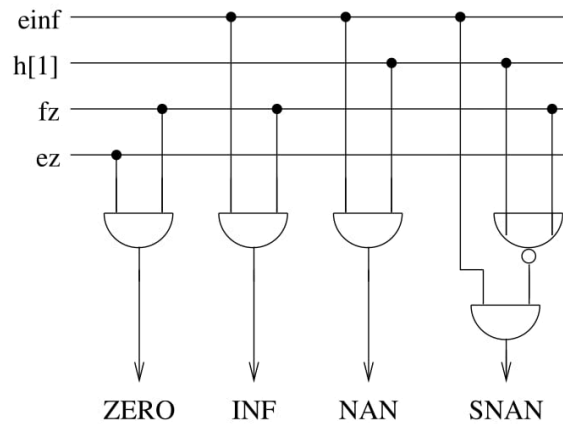


Figure 4: Special Cases

The SPECUNP circuit classifies the input operand using the exponent and fraction information produced by the unpacking stage. The following signals are used:

- *ez*: true when the exponent field is  $0^n$
- *einf*: true when the exponent field is  $1^n$
- *fz*: true when the fraction field is zero
- *h[1]*: used to distinguish quiet NaNs from signaling NaNs

Using these signals, the operand is classified as zero, infinity, quiet NaN, or signaling NaN. A zero is detected when both the exponent and fraction are zero. An infinity is detected when the exponent field is all ones and the fraction is zero. A NaN is detected when the exponent field is all ones and the fraction is not zero.

```

1  module exceptions (
2      input e_inf,
3      input h_1,
4      input fz,
5      input ez,
6      output ZERO,
7      output INF,
8      output NAN,
9      output SNAN
10 );
11 assign ZERO = fz & ez;
12 assign INF = e_inf & fz;
13 assign NAN = e_inf & h_1;
14 assign SNAN = e_inf & ~(h_1 | fz);

```

verilog

```
15 endmodule
```

The output flags generated by this module are passed together with the unpacked operand. Later stages use these flags to avoid treating special values as normal floating-point numbers.

### 3.3 NaN Selection

The `NANSELECT` circuit is used when an operation must return a quiet NaN. This can happen, for example, when one of the input operands is already a NaN or when an invalid operation is detected.

The selected NaN keeps the sign and payload bits from one of the input operands. The control signal  $NANa$  selects whether the NaN is taken from operand A or operand B. The most significant bit of the NaN payload is forced to one so that the produced NaN is quiet.

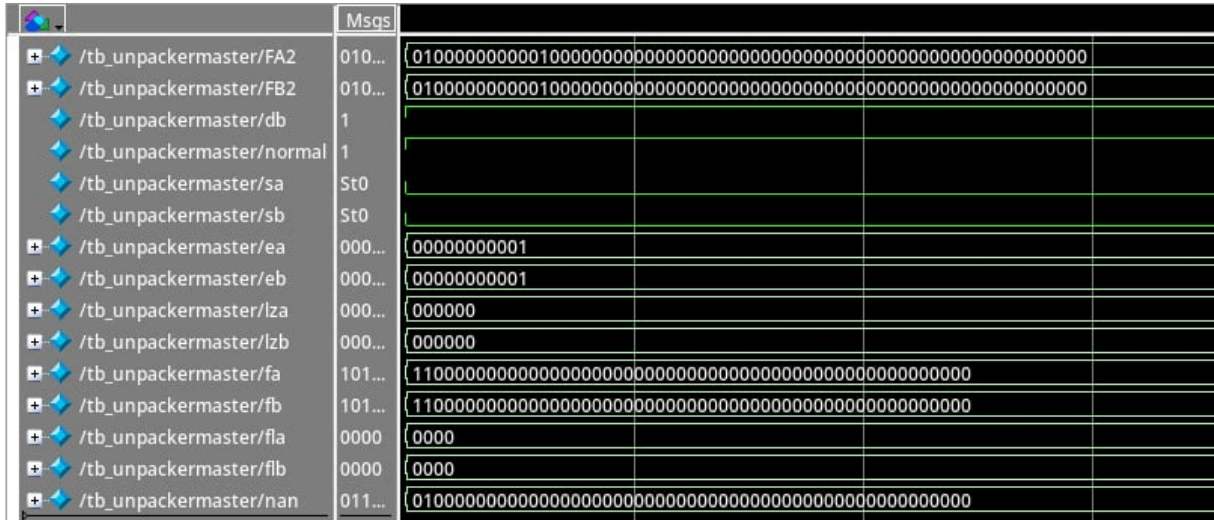
$$(s_{nan}, f_{nan}[0:51]) = \begin{cases} (s_a, 1, h_a[0:50]) & \text{if } NANa = 1 \\ (s_b, 1, h_b[0:50]) & \text{if } NANa = 0 \end{cases} \quad (23)$$

The corresponding SystemVerilog implementation is:

```
1  module nanselect(  
2      input sa,  
3      input [52:0] ha,  
4      input [52:0] hb,  
5      input sb,  
6      input nana,  
7      output snan,  
8      output [51:0] fnan  
9  );  
10  
11  assign snan = nana == 1 ? sa : sb;  
12  assign fnan = nana == 1 ? {1'b1, ha[50:0]} : {1'b1, hb[50:0]};  
13  
14  endmodule
```

verilog

### 3.4 Simulation



| Signal                    | Value  |
|---------------------------|--------|
| /tb_unpackermaster/FA2    | 010... |
| /tb_unpackermaster/FB2    | 010... |
| /tb_unpackermaster/db     | 1      |
| /tb_unpackermaster/normal | 1      |
| /tb_unpackermaster/sa     | St0    |
| /tb_unpackermaster/sb     | St0    |
| /tb_unpackermaster/ea     | 000... |
| /tb_unpackermaster/eb     | 000... |
| /tb_unpackermaster/lza    | 000... |
| /tb_unpackermaster/lzb    | 000... |
| /tb_unpackermaster/fa     | 101... |
| /tb_unpackermaster/fb     | 101... |
| /tb_unpackermaster/fla    | 0000   |
| /tb_unpackermaster/flb    | 0000   |
| /tb_unpackermaster/nan    | 011... |

Figure 5: Simulation of Unpacker

The simulation shows the unpacker outputs for two input operands, A and B. The signals *sa* and *sb* contain the sign bits. The signals *ea* and *eb* contain the converted exponents, and *fa* and *fb* contain the internal significands. The values *lza* and *lzb* store the leading-zero counts used during normalization.

The *fla* and *flb* contain the special-case flags generated by SPECUNP. These flags indicate whether each operand is zero, infinity, quiet NaN, or signaling NaN. In this example, both operands represent the double-precision value 3.0, so both sign bits are zero and neither operand is classified as a special value.

```

1  module tb_unpackermaster;
2
3      reg [63:0] FA2, FB2;
4      reg db, normal;
5      wire sa, sb;
6      wire [10:0] ea, eb;
7      wire [5:0] lza, lzb;
8      wire [52:0] fa, fb;
9      wire [3:0] fla, flb;
10     wire [52:0] nan;
11
12     unpackermaster uut (
13         .FA2(FA2),
14         .db(db),
15         .normal(normal),

```

verilog



## 4 Addition and Subtraction Unit

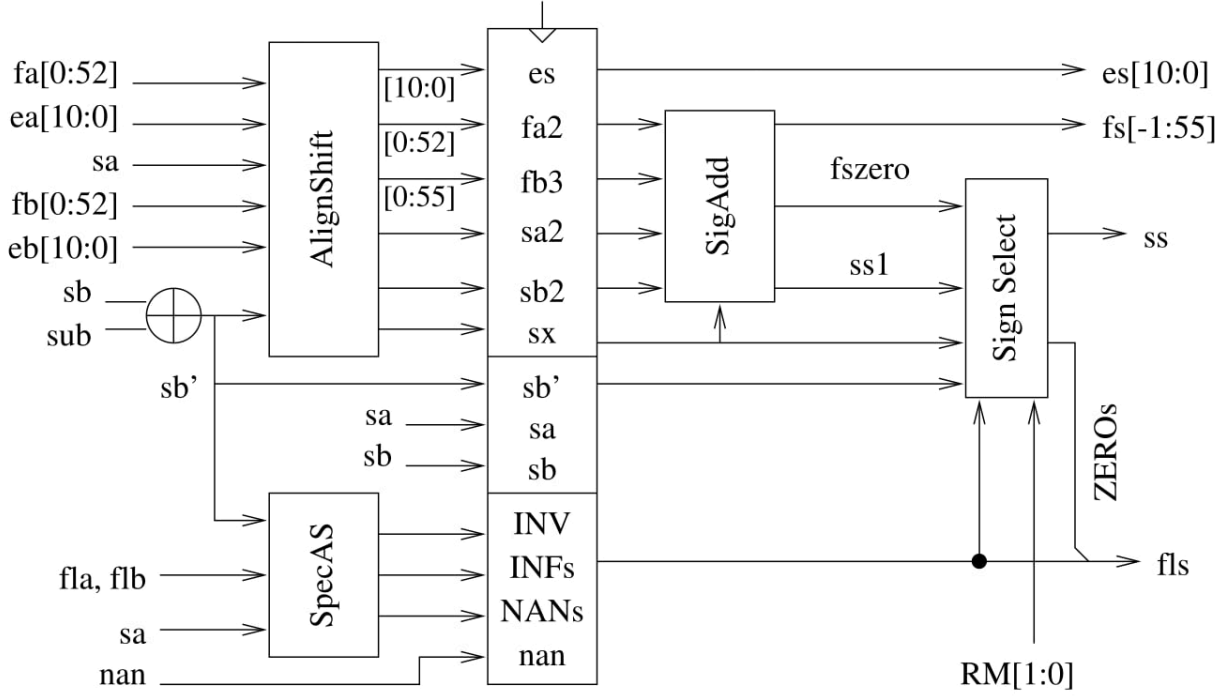


Figure 6: Overview of the adder/subtractor design

The addition/subtraction unit receives two unpacked operands from the unpacker. At this stage, the operands are already represented as  $(s, e, f)$ . The exponents are stored in two's complement form:

$$e_a = [e_a[n-1:0]], \quad e_b = [e_b[n-1:0]] \quad (24)$$

The significands are interpreted as

$$f_a = \langle f_a[0].f_a[1:p-1] \rangle, \quad f_b = \langle f_b[0].f_b[1:p-1] \rangle \quad (25)$$

Floating-point addition cannot be performed directly when the operands have different exponents. First, the significands must be aligned to the same exponent. Without loss of generality, assume that

$$\delta = e_a - e_b \geq 0 \quad (26)$$

Then the sum can be written as

$$\begin{aligned} S &= [[s_a, e_a, f_a]] + [[s_b, e_b, f_b]] \\ &= (-1)^{s_a} \cdot 2^{e_a} \cdot f_a + (-1)^{s_b} \cdot 2^{e_b} \cdot f_b \\ &= 2^{e_a} \cdot ((-1)^{s_a} \cdot f_a + (-1)^{s_b} \cdot 2^{-\delta} \cdot f_b). \end{aligned} \quad (27)$$

This form shows the main idea of the alignment stage. The larger exponent is factored out, and the significand belonging to the smaller exponent is shifted right by  $\delta$  positions. In the circuit, this is done by the **ALIGNSHIFT** block before the significands are added.

Subtraction is handled by converting it into addition with an inverted sign. Let  $a$ ,  $b$ , and  $b'$  be representable floating-point numbers with factorizations

$$(s_a, e_a, f_a), \quad (s_b, e_b, f_b), \quad (\overline{s_b}, e_b, f_b) \quad (28)$$

respectively. Then

$$\begin{aligned} a - b &= a - (-1)^{s_b} \cdot 2^{e_b} \cdot f_b \\ &= a + (-1)^{s_b} \cdot 2^{e_b} \cdot f_b \\ &= a + b'. \end{aligned} \quad (29)$$

## 4.1 Alignment Shift

Delta is already known as  $\delta = |e_a - e_b|$ . Since the following addition stage only needs a short representative of  $\delta$ , the circuit keeps that representative instead of the full exponent difference. The exact aligned significand would be

$$2^{-\delta} \cdot f_b \quad (30)$$

but the datapath replaces it with its short  $(p + 1)$ -bit representative

$$f' = [2^{-\delta} \cdot f_b]_{p+1} \quad (31)$$

This keeps enough information for the next stage while avoiding a large unbounded shift.

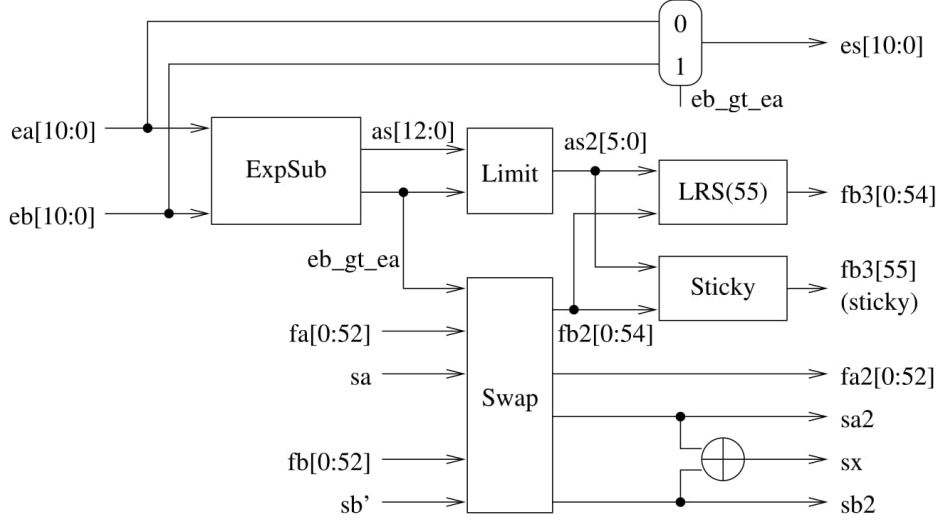


Figure 7: Circuit ALIGNSHIFT

The overall implementation details are not discussed here; the reader is referred to the implementation at (GitHub : Alignment Shift).

#### 4.1.1 EXPSUB

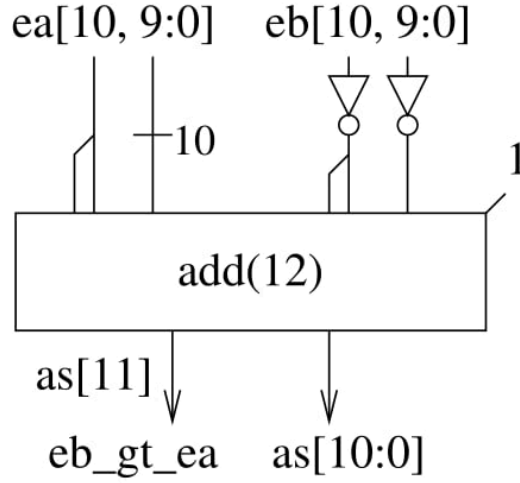


Figure 8: Circuit EXPSUB

EXPSUB computes the difference between the two exponents:

$$as = e_a - e_b \quad (32)$$

The sign of this difference determines which operand has the larger exponent. If the result is negative, then  $e_b > e_a$ , and the operands must be swapped before the alignment shift. The subtraction is implemented using the standard two's complement identity

$$e_a - e_b = e_a + \overline{e_b} + 1 \quad (33)$$

```

1  module exp_sub (
2      input [10:0] ea,
3      input [10:0] eb,
4
5      output eb_gt_ea,
6      output [10:0] as
7  );
8
9  wire [11:0] ina, inb;
10 wire [12:0] sum;
11
12 assign ina = {ea[10], ea[10:0]};
13 assign inb = {~eb[10], ~eb[10:0]};
14
15 parameter n = 12;
16 add #(n) ad(
17     .a(inb),
18     .b(ina),
19     .c_in(1'b1),
20     .sum(sum)
21 );
22
23 assign eb_gt_ea = sum[11];
24 assign as = sum[10:0];
25
26 endmodule

```

verilog

#### 4.1.2 LIMIT

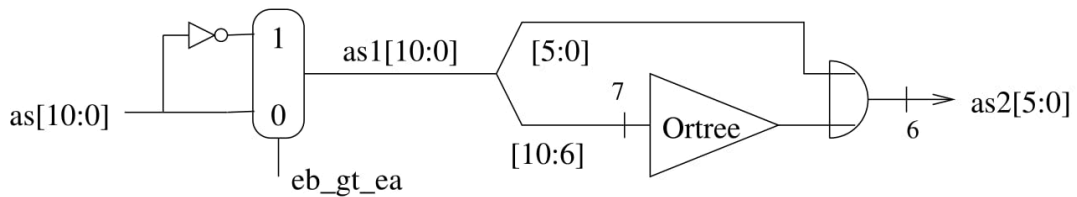


Figure 9: Circuit LIMIT

The LIMIT circuit computes a bounded version of the alignment shift distance. Let  $as[11:0]$  be the two's complement exponent difference produced by EXPSUB:

$$as = e_a - e_b \quad (34)$$

The circuit first forms an intermediate unsigned value  $as_1[10:0]$ :



$$as_1[10:0] = \begin{cases} as[10:0] & \text{if } as[11] = 0 \quad (e_a \geq e_b) \\ \frac{as[10:0]}{\delta} & \text{if } as[11] = 1 \quad (e_a < e_b) \end{cases} \quad (35)$$

Therefore, the value represented by  $as_1$  satisfies

$$\langle as_1 \rangle = \begin{cases} \delta & \text{if } (e_a \geq e_b) \\ \delta - 1 & \text{if } (e_a < e_b) \end{cases} \quad (36)$$

For  $e_a < e_b$ , the intermediate value becomes  $\delta - 1$ . This offset is intentional and is compensated later by the SWAP circuit.

In the second step, LIMIT bounds the shift distance. Since shifts beyond the needed representative width do not change the kept  $(p + 1)$ -bit result, the distance is saturated to a power of two minus one:

$$b = \lceil \log_2(p + 3) \rceil = 6, \quad B = 2^b - 1 = 63. \quad (37)$$

The limited shift amount  $as_2[5:0]$  is then

$$\langle as_2 \rangle = \min(\langle as_1 \rangle, B) = \begin{cases} B & \text{if } \langle as_1 \rangle \geq B \\ \langle as_1 \rangle & \text{otherwise} \end{cases} \quad (38)$$

```

1  module limit (
2      input  [10:0] as,
3      input          eb_gt_ea,
4      output [5:0]  as2
5  );
6
7  wire [10:0] as1;
8  wire or_out;
9
10 assign as1 = eb_gt_ea ? ~as : as;
11
12 parameter n = 5;
13 ortree #(n) or_tree (
14     .x(as1[10:6]),
15     .or_out(or_out)
16 );
17
18 assign as2 = as1[5:0] | or_out;
19
20 endmodule

```

verilog

#### 4.1.2.1 OR Tree.

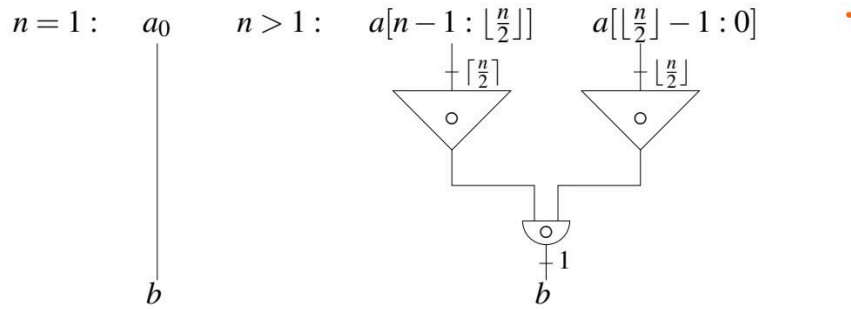


Figure 10: Circuit ORTREE computes OR of entire input recursively

```

1  module  ortree #(
2      parameter n = 2,
3      parameter npof2 = 2 ** $clog2(n)
4  )(
5      input [n-1:0] x,
6      output or_out
7  );
8      wire [npof2-1:0] padded_x;
9      assign padded_x = { {npof2 - n{1'b0}}, x};
10
11     generate
12         if (npof2 > 1) begin : recursive_case
13             wire L;
14             wire U;
15
16             ortree #(.n(npof2 / 2), .npof2(npof2 / 2)) lower_or (
17                 .x(padded_x[(npof2 / 2) - 1:0]),
18                 .or_out(L)
19             );
20
21             ortree #(.n(npof2 / 2), .npof2(npof2 / 2)) upper_or (
22                 .x(padded_x[npof2-1:(npof2 / 2)]),
23                 .or_out(U)
24             );
25
26             assign or_out = L | U;
27         end
28         else begin : base_case
29             assign or_out = padded_x[0];

```

```

30     end
31     endgenerate
32
33 endmodule

```

#### 4.1.3 Swapping Operands

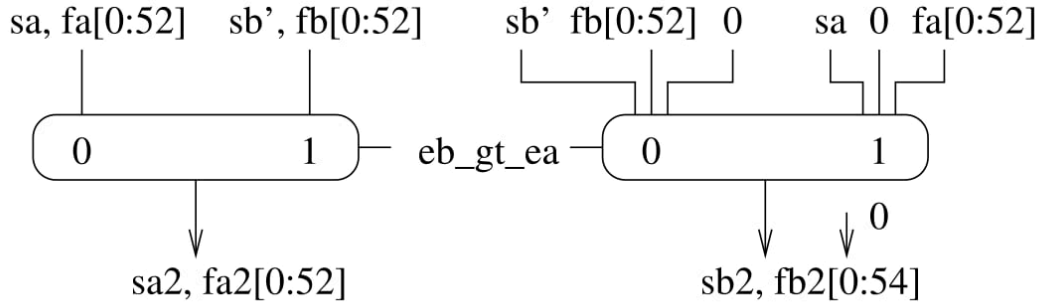


Figure 11: Circuit **SWAP**

After the exponent comparison, the operand with the larger exponent is placed on the main significand path. The operand with the smaller exponent is placed on the shifted path.

$$f_{a2}, f_{b2} = \begin{cases} (f_a, f_b) & \text{if } e_a \geq e_b \\ (f_b, f_a/2) & \text{if } e_a < e_b \end{cases} \quad (39)$$

The pre-shifted  $f_a/2$  is realized by appending a trailing zero bit and makes  $f_{b2}$  always 55 bits wide. This exactly compensates the  $\delta - 1$  approximation from **LIMIT**.

```

1  module swap (
2      input sa,
3      input [52:0] fa,
4      input sb,
5      input [52:0] fb,
6
7      input eb_gt_ea,
8
9      output sa2,
10     output [52:0] fa2,
11
12     output sb2,
13     output [54:0] fb2
14 );
15 assign sa2 = eb_gt_ea ? sb : sa;

```

verilog

```

16 assign fa2 = eb_gt_ea ? fb : fa;
17
18 assign sb2 = eb_gt_ea ? sa : sb;
19 assign fb2 = eb_gt_ea ? {1'b0, fa[52:0], 1'b0} : {fb[52:0], 2'b0};
20 endmodule

```

#### 4.1.4 Sticky Bit Computation

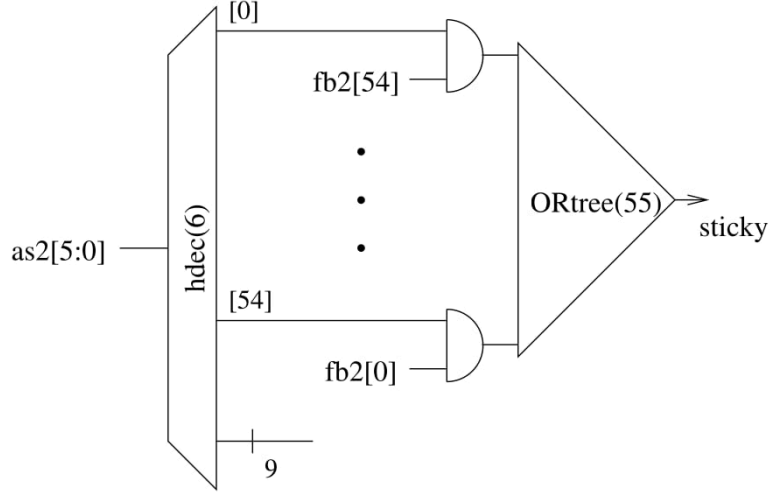


Figure 12: Circuit `STICKY`

After `SWAP` selects and pre-shifts the smaller significand, it produces  $f_{b2}[54:0]$ . Then `LRS` applies the bounded logical right shift by  $\langle as_2 \rangle$  bits. At this point, the alignment stage has kept the main  $(p+1)$ -bit representative, but one piece of information is still missing: whether any discarded bit was nonzero.

This is exactly the role of the sticky bit. It records whether the part of the shifted significand that lies to the right of the retained representative contains at least one 1.

Circuit `STICKY` computes

$$st = \bigvee_{i=p+2}^{54} f_{b2}[i + \langle as_2 \rangle] \quad (40)$$

In words,  $st$  is the logical OR of all shifted bits that fall strictly below position  $p+1$ . If any of those bits is set, the sticky bit becomes 1; otherwise, it remains 0.

The circuit implements this OR-reduction with a half-decoder. The lower bits of the shift amount select exactly the range of  $f_{b2}$  that can contribute to the sticky bit, and the final OR tree reduces that masked range to a single output.

```

1  module sticky (
2      input [5:0] as2,
3      input [54:0] fb2,
4      output sticky
5  );
6
7  parameter N = 6;
8  wire [63:0] y;
9
10 HDecJ #(.N(N)) hdec(
11     .x(as2),
12     .y(y)
13 );
14
15 wire [54:0] or_in;
16 assign or_in = fb2 & y[54:0];
17
18 parameter n = 55;
19 ortree #(n) or_tree(
20     .x(or_in),
21     .or_out(sticky)
22 );
23
24 endmodule

```

verilog

#### 4.1.5 Half Decoder

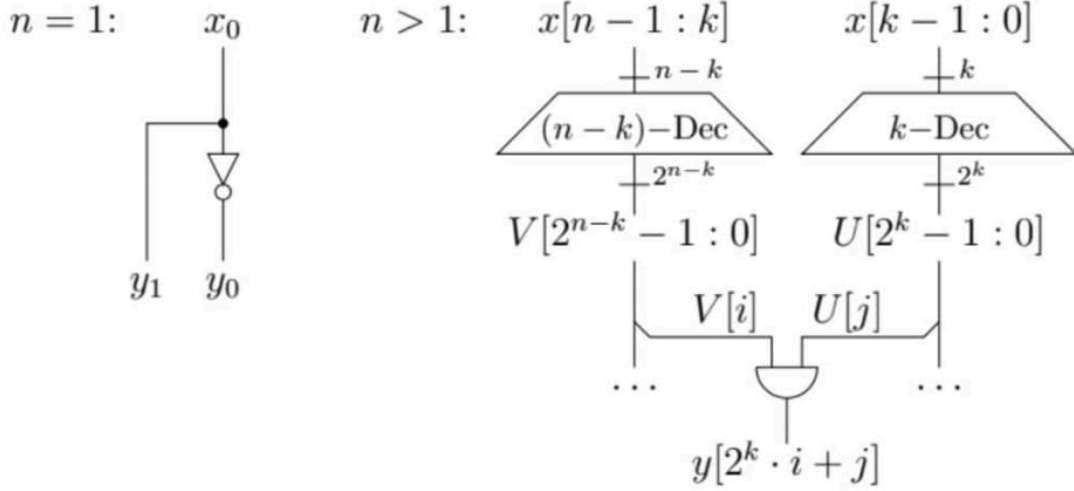


Figure 13: Circuit HALF\_DECODER

The half-decoder converts an  $n$ -bit binary input  $x[n-1:0]$  into a unary-style mask  $y[2^n-1:0]$ . Interpreting  $x$  as an unsigned integer, the output has  $\langle x \rangle$  active bits and the remaining bits cleared:

$$y = 0^{2^n - \langle x \rangle} 1^{\langle x \rangle} \quad (41)$$

This mask is useful because it turns a binary shift amount into a bit-vector that can directly select the portion of the operand affected by the shift.

A recursive construction is shown in Figure 13. Let  $k = \lceil \frac{n}{2} \rceil$ . The input is split into upper and lower parts, each decoded recursively. The full output is then assembled from the two smaller decoder outputs. For an output index written as  $i \cdot 2^k + j$ , the construction satisfies

$$\begin{aligned} y[i \cdot 2^k + j] = 1 &\leftrightarrow V[i] = 1 \wedge U[j] = 1 \text{ (construction)} \\ &\leftrightarrow \langle x[n-1:k] \rangle = i \wedge \langle x[k-1:0] \rangle = j \text{ (ind.hypothesis)} \\ &\leftrightarrow \langle x[n-1:0] \rangle = i \cdot 2^k + j. \end{aligned} \quad (42)$$

Thus, the recursive decoder produces the same unary-style mask as a direct decoder, but with a structured circuit that is easier to build compositionally.

```

1  module HDecJ #(
2      parameter N = 2,
3      parameter npof2 = 2 ** $clog2(N)
4  )(
5      input  [N-1:0] x,
6      output reg [2**N-1:0] y
7  );
8      wire [npof2-1:0] padded_x;
9      assign padded_x = { {npof2 - N{1'b0}}, x };
10
11     reg [npof2/2-1:0] x1;
12     reg [npof2/2-1:0] x2;
13     reg [2**(npof2/2)-1:0] U;
14     reg [2**(npof2/2)-1:0] V;
15
16     genvar i;
17     generate
18         if (npof2 > 1) begin : recursive_case
19             HDecJ #(
20                 .N(npof2 / 2),
21                 .npof2(npof2 / 2)
22             ) higher_decoder (
23                 .x(padded_x[npof2-1:npof2/2]),
24                 .y(U)
25             );
26
27             HDecJ #(
28                 .N(npof2 / 2),
29                 .npof2(npof2 / 2)
30             ) lower_decoder (
31                 .x(padded_x[npof2/2-1:0]),
32                 .y(V)
33             );
34             for (i = 0; i < 2**npof2; i = i + 1) begin : gen_indices
35                 localparam IL = i % (2**(npof2/2));
36                 localparam IH = i / (2**(npof2/2));
37                 assign y[i] = (IH == 0) ? (U[0] | V[IL]) : (U[IH] | (U[IH-1] &
38                     V[IL]));
39             end

```

```

40     end
41     else begin : base_case
42         assign y[0] = padded_x[0];
43         assign y[1] = 0;
44     end
45 endgenerate
46
47 endmodule

```

## 4.2 Significand Add

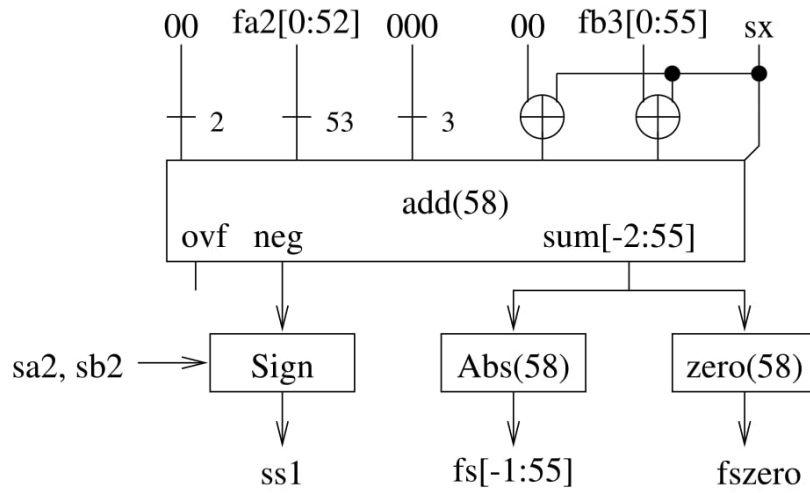


Figure 14: Circuit SIGADD

At this point, the exponent mismatch has already been handled by the alignment stage. The larger significand  $f_{a2}[0 : 52]$  and the shifted short representative of the smaller significand  $f_{b3}[0 : 55]$  now share a common scale. What remains is to combine them with the correct signs.

Circuit SIGADD performs this signed significand addition. Depending on the operation and the operand signs, the aligned smaller significand is either added directly or subtracted:

$$sum = f_{a2} + (-1)^{s_x} \cdot f_{b3} \quad (43)$$

Here,  $s_x$  is the add/subtract control signal:  $s_x = 0$  selects addition, and  $s_x = 1$  selects subtraction.

The circuit implements both cases with a single wide two's-complement adder. Both operands are extended to 58 bits. For subtraction, the bits of  $f_{b3}$  are inverted



and the carry-in is forced to 1, which forms the two's-complement negation of the second operand. After the addition, the result may be positive, negative, or zero. The sign bit of the intermediate sum tells whether the magnitude must be negated. The ABS circuit converts the two's-complement result into a positive significand magnitude, fszero records whether the result is exactly zero, and ss1 selects the sign that must be carried into the normalization stage.

```

1  module sigadd(
2      input [52:0] fa2,
3      input [55:0] fb3,
4      input  sa2,
5      input  sb2,
6      input  sx,
7      output [56:0] fs,
8      output fszero,
9      output ss1
10 );
11
12 wire [57:0] a, b;
13 wire [58:0] sum;
14
15 assign a = {2'b00, fa2[52:0], 3'b000};
16 assign b = {2'b00^{2{sx}}, fb3^{56{sx}}};
17
18 parameter n = 58;
19 add #(n) ad(
20     .a(a),
21     .b(b),
22     .c_in(sx),
23     .sum(sum)
24 );
25
26 wire [57:0] res;
27 assign res = sum[57:0];
28 assign fszero = (res == 58'b0);
29 assign neg = res[57];
30 assign ss1 = (sb2 & neg) | (sa2 & ~(sb2 & neg));
31
32
33

```

verilog

```

34 abs ut(
35     .x(res),
36     .abs(fs)
37 );
38
39 endmodule

```

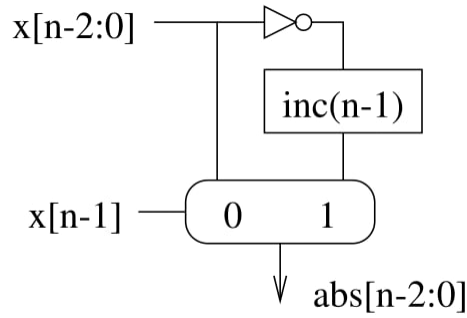


Figure 15: Circuit **ABS**

The adder output is a signed two's-complement value, but the following stages operate on a nonnegative significand magnitude. Circuit **ABS** performs this conversion. For an  $n$ -bit two's-complement input  $s[n-1:0]$ , the most significant bit determines whether the value is already nonnegative or must be negated:

$$abs[n-1:0] = \begin{cases} s[n-1:0] & \text{if } s[n-1] = 0 \\ -s[n-1:0] & \text{if } s[n-1] = 1 \end{cases} \quad (44)$$

```

1 module abs#(parameter N=58)(
2   input  [N-1:0] x,
3   output [N-2:0] abs
4 );
5
6 assign abs = x[N-1] ? (~x[N-2:0] + 1) : x[N-2:0];
7
8 endmodule

```

verilog

### 4.3 Special Cases

The circuit SPECAS handles IEEE-754 special cases and detects invalid operations. Using the eight input flags for each operand, it produces three output flags:

- INFs: indicates an infinite result,
- NANs: signals a quiet NaN result,
- INV: flags an invalid operation.

The logic follows the IEEE rules exactly:

$$\begin{aligned} INV &= (SNANa \vee SNANb) \vee (INFa \wedge INFb \wedge (s_a \oplus s_b)) \\ NANs &= INV \vee (NANa \vee NANb) \\ INFs &= (INFa \vee INFb) \wedge \overline{NANs} \end{aligned} \tag{45}$$

```
1  module spec(  
2      input sb,  
3      input sa,  
4      input [3:0] fla,    // [Zeroa, INFa, SNANa, NANa]  
5      input [3:0] flb,    // [Zerob, INFb, SNbNb, NbNb]  
6      output INFs,  
7      output NANs,  
8      output INV  
9  );  
10  
11 assign INV = (fla[1] | flb[1]) | (fla[2] & flb[2] & (sa ^ sb));  
12 assign NANs = INV | (fla[0] | flb[0]);  
13 assign INFs = (fla[2] | flb[2]) & ~NANs;  
14  
15 endmodule
```

verilog

## 4.4 Sign Computation

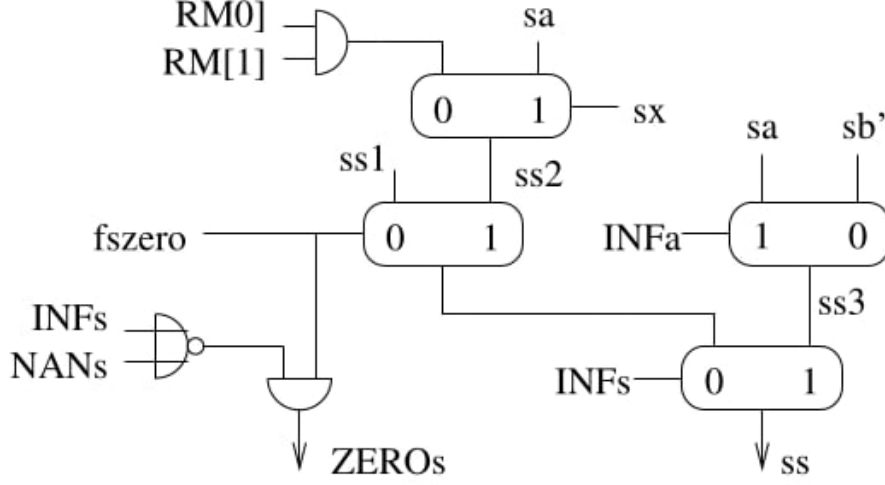


Figure 16: Circuit SIGNSELECT

**SIGNSELECT** is the final stage of the sign computation pipeline. By this point, the arithmetic datapath has already produced a preliminary result sign  $ss_1$  and a zero flag  $f_z$ . However, this preliminary sign is only valid for ordinary finite, nonzero results. Zero, infinity, and NaN cases require separate handling.

The circuit receives the preliminary sign  $ss_1$  from **SIGADD**, the zero flag  $f_z$ , the infinity flag  $INF_s$ , the NaN flag  $NaN$ , the operand signs  $s_a$  and  $s_b$ , the control signal  $s_x$ , and the rounding mode  $RM[1:0]$ . It produces the final result sign  $s_s$  and the true zero-result flag  $ZERO$ .

For finite nonzero results, no correction is needed: the final sign is simply the preliminary sign computed by **SIGADD**.

For infinite results, the arithmetic sign is ignored. Instead, the result keeps the sign of the infinite operand. If operand **a** is infinite, its sign is used; otherwise, the sign of operand **b** is used:

$$s_{s3} = \begin{cases} s_a & \text{if } INF_a \\ s_b & \text{if } INF_b \wedge \neg INF_a \end{cases} \quad (46)$$

For zero results, the sign is selected by a special zero-sign rule. According to the implementation,

$$s_{s2} = \begin{cases} 0 & \text{if } s_x \wedge (\neg(RM[1] \vee RM[0])) \\ 1 & \text{if } s_x \wedge (RM[1] \vee RM[0]) \\ s_a & \text{if } \overline{s_x} \end{cases} \quad (47)$$

Thus, when  $s_x$  is set, the zero result keeps the sign of operand **a**. Otherwise, the sign is determined by the rounding mode. In this encoding,  $RM[0] \wedge RM[1]$  selects the negative zero case.

$$s_s = \begin{cases} s_{s3} & \text{if } INF_s \\ s_{s2} & \text{if } \neg INF_s \wedge (f_s = 0) \\ s_{s1} & \text{if } \neg INF_s \wedge (f_s \neq 0) \end{cases} \quad (48)$$

The **ZERO** output marks a true zero result. A result is considered zero only when the significand result is zero and the final value is neither infinite nor NaN:

$$ZERO_s = f_{szero} \wedge \neg(INF_s \vee NAN_s). \quad (49)$$

```

1  module sign_select(
2      input [1:0] RM,
3      input      fz,
4      input      sa,
5      input      sx,
6      input      sb,
7      input      ss1,
8      input      INFa,
9      input      INFs,
10     input      NAN,
11     output     ZERO,
12     output     ss
13 );
14
15 wire ss2, ss3;
16
17 assign ss2 = sx ? sa : RM[0] & RM[1];
18 assign ss3 = INFa ? sa : sb;
19 assign ss = INFs ? ss3 : (fz ? ss2 : ss1);
20 assign ZERO = ~(INFs | NAN) & fz;
21
22 endmodule

```

verilog

## 5 Multiplication and Division Unit

The multiplication and division unit implements the arithmetic operations that produce wider intermediate significand results. This chapter first describes the multiplier, whose structure is built from a small number of repeated ideas: generate partial products, reduce them with an addition tree, and finish with a carry-look-ahead adder.

The multiplier is organized as a partial-product generator followed by a Wallace-style reduction tree. The tree reduces the partial products to two vectors,  $s$  and  $t$ , whose numeric sum equals the full product sum. A final carry-look-ahead adder computes  $s + t$  and produces the binary product.

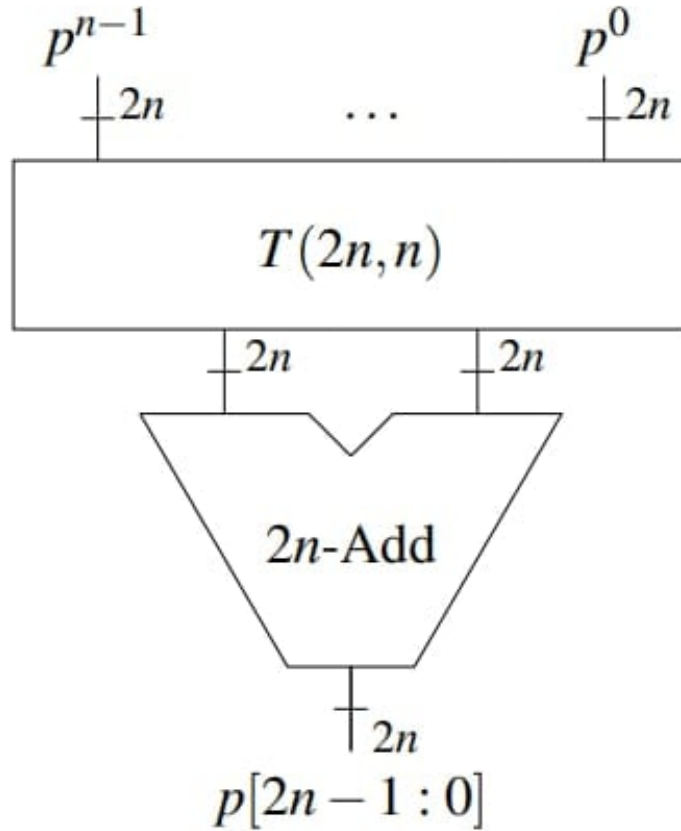


Figure 17: Wallace tree

An  $n$ -multiplier is a circuit

$$M_n : B^n \times B^n \rightarrow B^{2n} \quad (50)$$

with inputs  $a, b \in B^n$  and output  $p \in B^{2n}$  such that

$$\langle a \rangle \cdot \langle b \rangle = \langle p \rangle \quad (51)$$

The design consists of three stages (see Figure 17):

1. Partial-product generation

For each  $i \in [n - 1 : 0]$  we form the  $i$ -th partial product

$$p^i = 0^{n-i-1}(a \wedge b_i)0^i. \quad (52)$$

This produces  $n$  partial products  $p^0, \dots, p^{n-1}$ , each of width  $2n$ .

2. Wallace-tree

The reduction tree  $T(2n, n)$  compresses the partial products into two operands  $s$  and  $t$  such that

$$\langle s \rangle + \langle t \rangle = \sum_{i=0}^{n-1} \langle p^i \rangle \pmod{2^{2n}} \quad (53)$$

3. Final carry-look-ahead addition

The final product is computed as

$$\langle p[2n - 1 : 0] \rangle = \langle s \rangle + \langle t \rangle \quad (54)$$

The following System Verilog implementation follows this structure:

```

1  module multree(
2      input  [57:0]    a,
3      input  [57:0]    b,
4      output [115:0]   out
5  );
6
7  wire [115:0] partials [57:0];
8  wire [115:0] temp [57:0];
9  wire [115:0] t,s;
10
11
12  genvar i, j;
13  generate
14      for (i = 0; i < 58; i++) begin
15          assign temp[i][115:58] = 0;
16          for (j = 0; j < 58; j++) begin
17              assign temp[i][j] = a[i] & b[j];
18          end
19      end

```

verilog

```

20
21     for(i=0; i < 58; i++) begin
22         assign partials[i] = temp[i] << i;
23     end
24 endgenerate
25
26 parameter n = 58;
27
28 ftaddrec #(n) ftadd(
29     .partials(partial),
30     .t(t),
31     .s(s)
32 );
33
34 parameter m = 116;
35 wire [116:0] sum;
36
37 add #(m) ad(
38     .a(t),
39     .b(s),
40     .c_in(1'b0),
41     .sum(sum)
42 );
43
44
45 assign out = sum[115:0];
46
47 endmodule

```



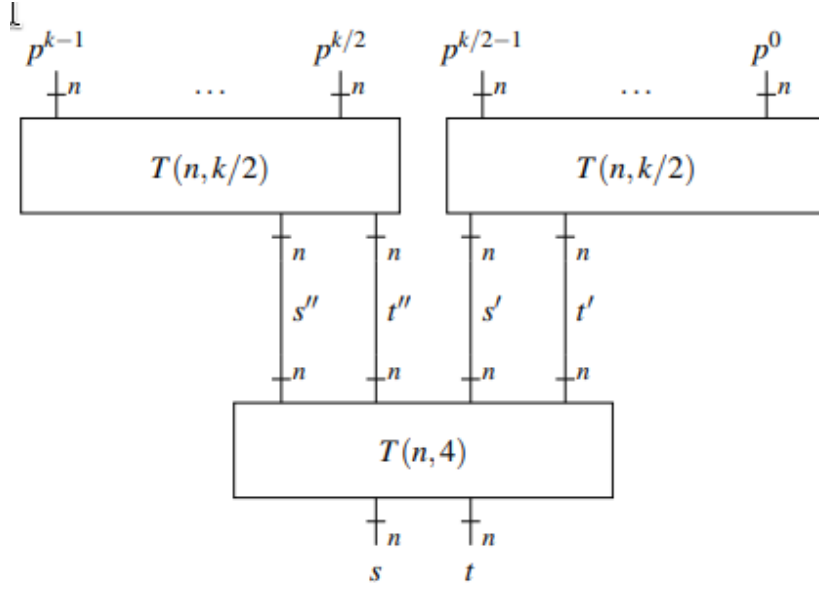


Figure 18: (n,k)-addition trees

The core of the design is the addition tree  $T(116, 58)$ , which reduces the 58 partial products into two vectors  $s$  and  $t$  satisfying

The tree is implemented recursively that reduces groups of partial products and combines intermediate results using full adders.

$$\langle s \rangle + \langle t \rangle = \sum_{i=0}^{57} \langle p^i \rangle \pmod{2^{116}}. \quad (55)$$

```

1  module ftaddrec #(
2      parameter n = 58,
3      parameter npof = 2 ** $clog2(n)
4  )
5  (
6      input [115:0] partials[n-1:0],
7      output [115:0] t,
8      output [115:0] s
9  );
10
11 wire [115:0] sum1, sum2, sum3, sum4;
12 wire [116:0] carry, sum;
13
14 wire [115:0] partials1 [npof-1:0];
15
16 genvar j;
17 generate
18     for (j = 0; j < npof; j = j + 1) begin

```

verilog

```

19         assign partials1[j] = (j < n) ? partials[j] : 0;
20     end
21 endgenerate
22
23 function automatic [1:0] full_adder;
24     input logic a;
25     input logic b;
26     input logic c;
27
28     logic c_p;
29     logic s_p;
30
31     begin
32         c_p = (a & b) | (a & c) | (b & c);
33         s_p = a ^ b ^ c;
34
35         full_adder = {c_p, s_p};
36     end
37 endfunction
38
39 genvar i;
40 generate
41 if (npof > 4) begin
42
43     ftaddrec #(.n(npof/2), .npof(npof / 2)) zur1(
44         .partials(partials1[npof/2-1:0]),
45         .t(sum1),
46         .s(sum2)
47     );
48
49     ftaddrec #(.n(npof/2), .npof(npof / 2)) zur2(
50         .partials(partials1[npof-1:npof/2]),
51         .t(sum3),
52         .s(sum4)
53     );
54
55     assign carry[0] = 0;
56
57     for (i = 0; i < 116; i = i + 1) begin
58

```

```

59     wire [1:0] sum5 = full_adder(sum1[i], sum2[i], sum3[i]);
60
61     assign sum[i] = sum5[0];
62     assign carry[i+1] = sum5[1];
63 end
64
65 assign sum[116] = 0;
66 assign t[0] = 0;
67
68 for (i = 0; i < 115; i = i + 1) begin
69
70     wire [1:0] sum6 = full_adder(sum[i], carry[i], sum4[i]);
71
72     assign s[i] = sum6[0];
73     assign t[i+1] = sum6[1];
74 end
75
76 wire [1:0] sum6_last = full_adder(sum[115], carry[115], sum4[115]);
77
78 assign s[115] = sum6_last[0];
79
80 end else begin : Base
81
82     assign carry[0] = 0;
83
84     for (i = 0; i < 116; i = i + 1) begin
85
86         wire [1:0] sum5 = full_adder(
87             partials1[0][i],
88             partials1[1][i],
89             partials1[2][i]
90         );
91
92         assign sum[i] = sum5[0];
93         assign carry[i+1] = sum5[1];
94     end
95
96     assign sum[116] = 0;
97     assign t[0] = 0;
98

```

```

99     for (i = 0; i < 115; i = i + 1) begin
100         wire [1:0] sum6 = full_adder(sum[i], carry[i], partials1[3][i]);
101
102         assign s[i] = sum6[0];
103         assign t[i+1] = sum6[1];
104     end
105
106     wire [1:0] sum6_last = full_adder(sum[115], carry[115], partials1[3][115]);
107
108     assign s[115] = sum6_last[0];
109
110 end
111 endgenerate
112
113 endmodule

```

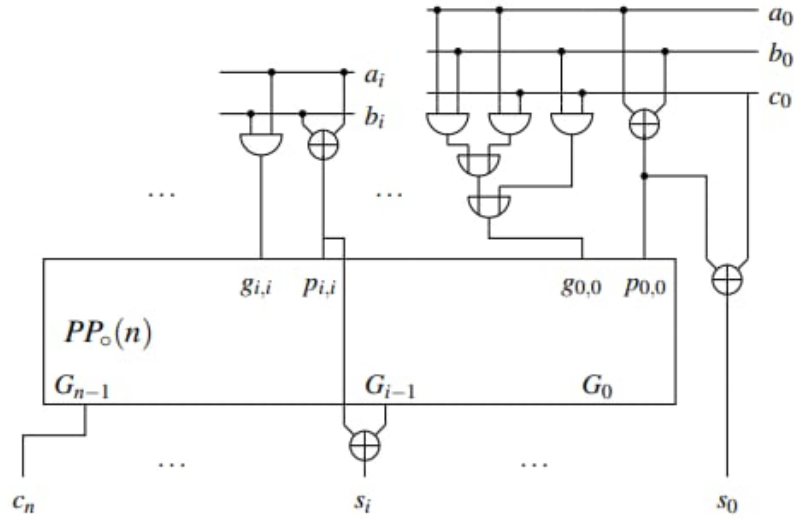


Figure 19: Carry-look-ahead adder

For operands  $a[n-1:0], b[n-1:0] \in \mathbb{B}^n$  and carry-in  $c_0 \in \mathbb{B}$ , the carry-look-ahead adder produces sum  $s[n-1:0]$  and carry-out  $c_n$ .

For indices  $i \geq j$  we define propagate and generate signals

$$p_{i,j}(a, b) \equiv \langle a[i:j] \rangle + \langle b[i:j] \rangle = \langle 1^{i-j+1} \rangle \quad (56)$$

$$g_{i,j}(a, b) \equiv \begin{cases} \langle a[i:j] \rangle + \langle b[i:j] \rangle \geq \langle 10^{i-j+1} \rangle & j > 0 \\ \langle a[i:j] \rangle + \langle b[i:j] \rangle + c_0 \geq \langle 10^{i-j+1} \rangle & j = 0 \end{cases} \quad (57)$$

Carry computation is

$$c_{i+1} = g_i \vee (p_i \wedge c_i) \quad (58)$$

For single bits these simplify to

$$p_{i,i} = a_i \oplus b_i \quad (59)$$

$$g_{i,i} = \begin{cases} a_i \wedge b_i & i > 0 \\ a_0 \wedge b_0 \vee a_0 \wedge c_0 \vee b_0 \wedge c_0 & i = 0. \end{cases} \quad (60)$$

The propagate and generate signals for adjacent intervals can be combined associatively and consider indices  $k$  splitting interval  $[j : i]$  of indices into two adjacent intervals  $[j : k]$  and  $[k + 1 : i]$ :

$$\begin{aligned} p_{i,j} &= p_{i,k+1} \wedge p_{k,j}, \\ g_{i,j} &= g_{i,k+1} \vee g_{k,j} \wedge p_{i,k+1}. \end{aligned} \quad (61)$$

```

1  module add #(parameter n = 10) (
2      input  [n-1:0] a,
3      input  [n-1:0] b,
4      input          c_in,
5      output [n:0]   sum
6  );
7
8  wire [n-1:0] p, g;
9  wire [n:0]   c;
10
11 assign c[0] = c_in;
12
13 genvar i;
14 generate
15 for (i = 0; i < n; i = i + 1) begin : pg_calc
16     assign g[i] = a[i] & b[i];
17     assign p[i] = a[i] ^ b[i];
18 end
19 endgenerate
20
21 generate
22 for (i = 0; i < n; i = i + 1) begin : carry_calc
23     assign c[i+1] = g[i] | (p[i] & c[i]);
24 end
25 endgenerate
26
27 generate
28 for (i = 0; i < n; i = i + 1) begin : sum_calc

```

verilog

```

29     assign sum[i] = p[i] ^ c[i];
30 end
31 endgenerate
32
33 assign sum[n] = c[n];
34
35 endmodule

```

## 5.1 Sign and Exponent Computation

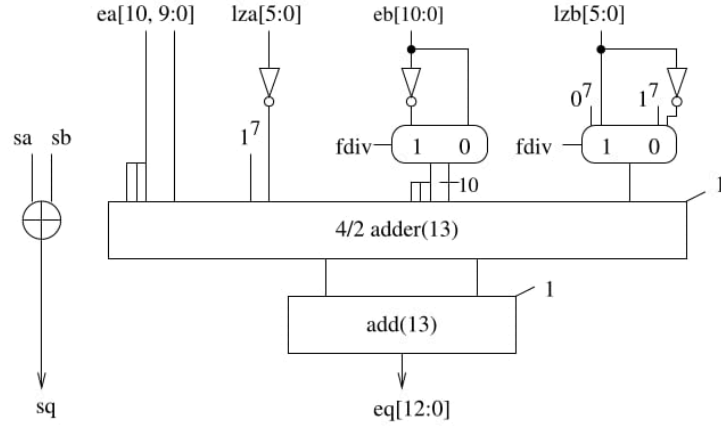


Figure 20: SIGN/EXPMD

The sign of the result is computed independently using a single exclusive-or operation:

$$s_q = s_a \oplus s_b \quad (62)$$

The exponent computation is controlled by the signal *fdiv*, which distinguishes between multiplication and division modes. Depending on the operation, the exponent is computed as:

$$e_q = \begin{cases} e_a - lza + (e_b - lzb) & \text{if } \neg \text{fdiv} \\ e_a - lza - (e_b - lzb) & \text{if } \text{fdiv} \end{cases} \quad (63)$$

where  $e_a$  and  $e_b$  are the unpacked exponents, and  $lza$  and  $lzb$  represent normalization shifts (leading-zero adjustments).

To efficiently implement this computation, the circuit uses a carry-save reduction stage based on a 4/2 adder. This unit allows multiple operands to be combined without immediate carry propagation, significantly reducing the critical path delay.

The inputs to the reduction stage include:

- exponent  $e_a$ ,
- exponent  $e_b$  (added or subtracted depending on `fdiv`),
- normalization corrections  $lz_a$  and  $lz_b$ ,
- constant terms required for alignment.

The intermediate result produced by the 4/2 adder is then passed to a final carry-propagate adder, which generates the resulting exponent  $e_q$ .

The exponent range can be estimated conservatively, which motivates the use of  $(n + 2)$ -bit two's complement representation to avoid overflow during intermediate computations.

```

1  module signexpmd(
2      input sa,
3      input [10:0] ea,
4      input [5:0] lza,
5
6      input sb,
7      input [10:0] eb,
8      input [5:0] lzb,
9
10     input fdiv,
11
12     output sq,
13     output [12:0] eq
14 );
15
16 wire [12:0] a, b, c, d;
17 wire [13:0] t, s;
18
19 assign sq = sa ^ sb;
20
21 assign a = {ea[10], ea[10], ea[10:0]};
22 assign b = {7'b1111111, ~lza[5:0]};
23 assign c = fdiv ? {~eb[10], ~eb[10], ~eb[10:0]} : {eb[10], eb[10], eb[10:0]};
24 assign d = fdiv ? {7'b0, lzb[5:0]} : {7'b1111111, ~lzb[5:0]};
25
26 ftadd add(
27     .a(a),
28     .b(b),
29     .c(c),

```

verilog

```

30     .d(d),
31     .t(t),
32     .s(s)
33 );
34
35 wire [13:0] sum;
36
37 parameter n = 13;
38 add #(n) ad(
39     .a(t[12:0]),
40     .b(s[12:0]),
41     .c_in(1'b1),
42     .sum(sum)
43 );
44
45 assign eq = sum[12:0];
46
47 endmodule

```

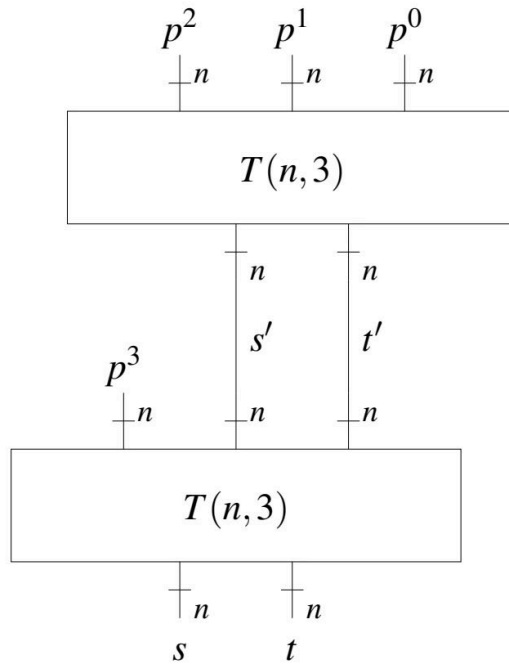


Figure 21:  $(n,4)$ -addition tree from two  $(n,3)$ -addition trees



We have implemented  $4/2$  adder(13) with  $(n, 4)$ -addition tree.

```
1  module ftadd #(parameter n = 13) (  
2      input [n-1:0] a,  
3      input [n-1:0] b,  
4      input [n-1:0] c,  
5      input [n-1:0] d,  
6      output [n:0] t,  
7      output [n:0] s  
8  );  
9  
10 wire [n:0] t1, s1;  
11  
12 three2add #(n) stage1 (  
13     .a(a),  
14     .b(b),  
15     .c(c),  
16     .c_in(1'b1),  
17     .t(t1),  
18     .s(s1)  
19 );  
20  
21 three2add #(n) stage2 (  
22     .a(s1[n-1:0]),  
23     .b(t1[n-1:0]),  
24     .c(d),  
25     .c_in(1'b0),  
26     .t(t),  
27     .s(s)  
28 );  
29  
30 endmodule
```

verilog

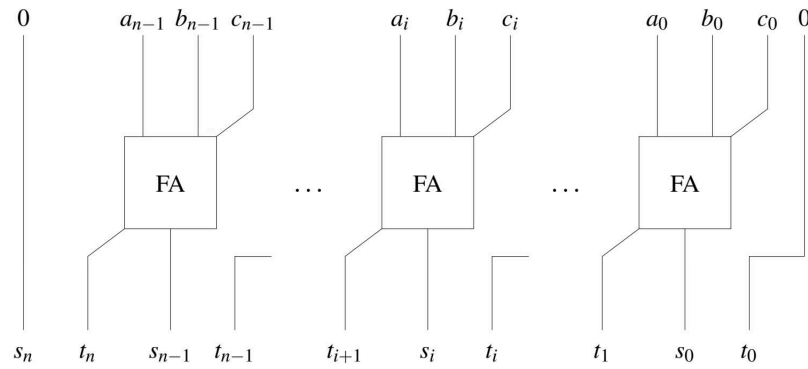


Figure 22: n-carry-save adder

Each (n,3)-addition tree consists of n full adders in parallel form an n-carry-save adder.

```

1  module three2add #(parameter n = 11) (
2      input [n-1:0] a,
3      input [n-1:0] b,
4      input [n-1:0] c,
5      input      c_in,
6      output [n:0] t,
7      output [n:0] s
8  );
9  genvar i;
10 assign t[0] = c_in;
11
12 generate
13 for (i = 0; i < n; i = i + 1) begin : add_bits
14
15     // full adder computation
16     wire c_p = (a[i] & b[i]) | (a[i] & c[i]) | (b[i] & c[i]);
17     wire s_p = a[i] ^ b[i] ^ c[i];
18
19     assign s[i] = s_p;
20     assign t[i+1] = c_p;
21 end
22 endgenerate
23
24 assign s[n] = 0;
25 endmodule

```

verilog

## 5.2 Multiplication and Division of significands

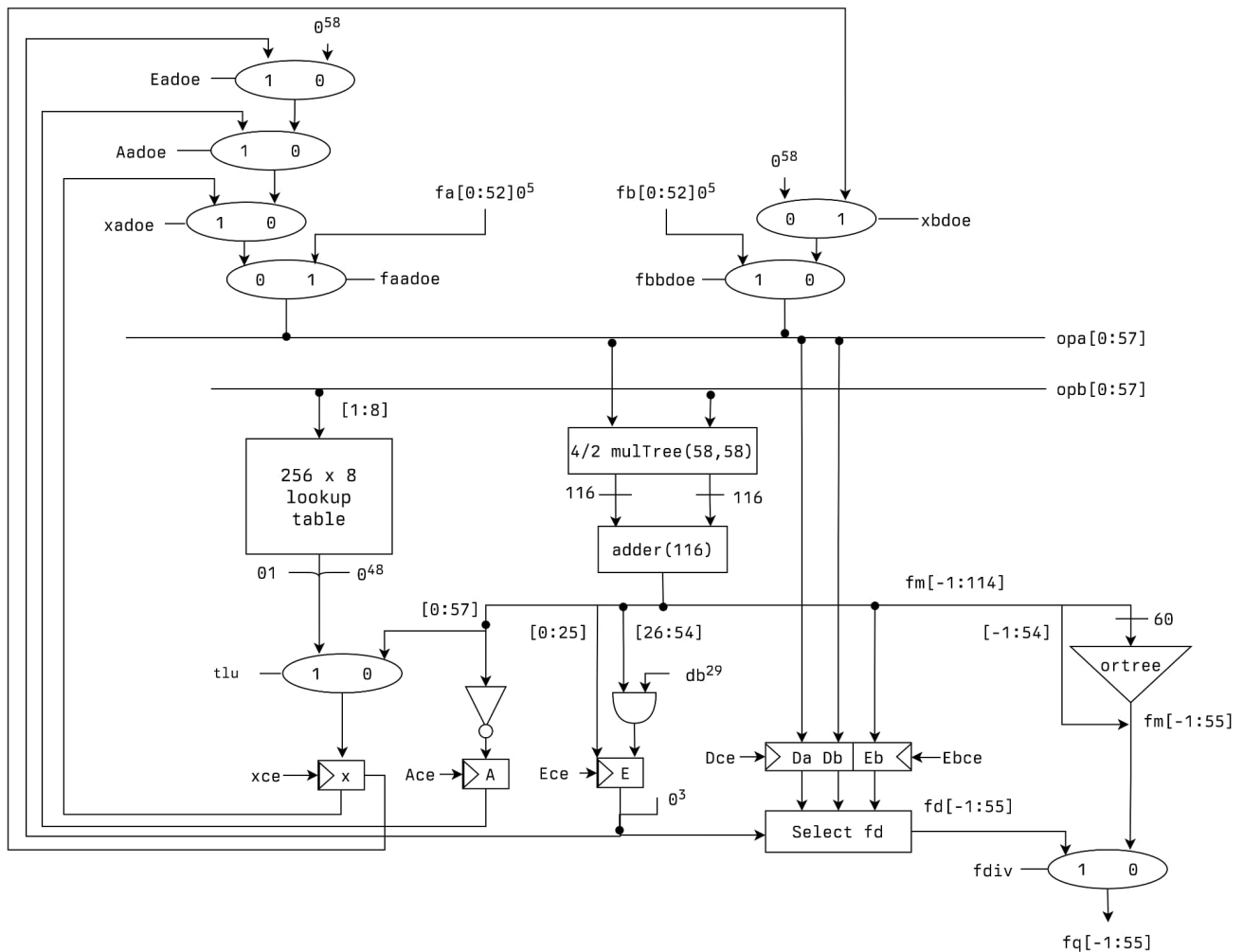


Figure 23: SIGFMD

The **UNPACKER** provides normalized floating-point operands to the multiply/divide unit. While multiplication of significands follows directly from the structure described earlier, division requires additional processing, since the quotient must be computed, normalized, and then correctly rounded.

Let  $a$  and  $b$  be finite, non-zero floating-point numbers in normalized form:

$$\begin{aligned}\hat{\eta}(a) &= (s_a, e_a - lz_a, f_a) \\ \hat{\eta}(b) &= (s_b, e_b - lz_b, f_b).\end{aligned}\tag{64}$$

The significands satisfy  $f_a, f_b \in [1, 2)$ .

The division data path computes a correctly rounded result  $r(a/b)$  by separating the computation into three parts: sign, exponent, and significand. The sign and exponent are handled in the previous stage; this section focuses on the significand path.

In the textbook design, several intermediate values are connected through shared tri-state buses controlled by output-enable signals. In the SystemVerilog implementation, these tri-state buses were replaced by explicit multiplexer trees. The control signals still follow the textbook naming, but instead of enabling a value onto a shared bus, they select the corresponding source for the multiplier and register inputs. This preserves the behavior of the original data path while using a structure that maps directly to synthesizable hardware.

### 5.2.1 Quotient structure

The first step is to compute the raw quotient of the significands:

$$q = \frac{f_a}{f_b} \in (1/2, 2) \quad (65)$$

Since both operands lie in  $[1, 2)$ , the quotient is guaranteed to lie in a bounded range and may require renormalization before rounding. At this stage, the division unit produces an intermediate representation

$$a/b = \llbracket [s_q, e_q, q] \rrbracket \quad (66)$$

where

$$s_q = s_a \oplus s_b \quad \text{and} \quad e_q = e_a - e_b. \quad (67)$$

Rather than directly finalizing  $q$ , the hardware computes a rounded significand  $f_d$  that will be used by the rounding unit together with  $(s_q, e_q)$ . The key task of the significand division unit is therefore to compute an accurate approximation of  $1/f_b$  and use it to form  $q$  via multiplication.

### 5.2.2 Reciprocal approximation

Instead of performing direct division, the unit computes an approximation of  $1/f_b$  using a lookup table followed by iterative refinement. The unpacker provides a normalized representation of  $f_b$ :

$$f_b = \langle 1.f_b[1 : p-1] \rangle \in [1, 2) \quad (68)$$

The interval  $[1, 2)$  is partitioned into  $2^\gamma$  sub-intervals of the form

$$[1 + t \cdot 2^{-\gamma}, 1 + (t+1) \cdot 2^{-\gamma}). \quad (69)$$

Each sub-interval is represented by a midpoint derived from the leading bits of  $f_b$ . The midpoint of the interval containing  $f_b$  is therefore

$$f_{b'} = \langle 1.f_b[1 : \gamma]1 \rangle \quad (70)$$

From this midpoint, we define an exact reciprocal value

$$x' = 1/f_{b'} \quad (71)$$

However, instead of storing this exact value, we store a simplified version that fits into fixed-point format. This is done by rounding  $x'$  to the nearest multiple of  $2^{-\gamma-1}$ . If two values are equally close, the higher one is chosen. This rule defines the entry stored in the ROM for that interval.

As a result, the initial approximation is stored in fixed-point form as

$$x_0 = \langle 0.x_0[1 : \gamma + 1] \rangle \quad (72)$$

This approximation lies in the interval  $(\frac{1}{2}, 1)$  and is used as the starting point for Newton–Raphson refinement.

We are generating initial approximations in Rust and are stored in a file.

```

1  use std::fs::File;
2  use std::io::{Result, Write};
3
4  pub fn lookup_table() -> Result<()> {
5      let mut file = File::create("lookup_table.txt")?;
6
7      for fb in 0..=255 {
8          let sig = 1.0 + (fb as f64) / 256.0; // significant in [1,2)
9
10         let recip = 1.0 / sig; // reciprocal in (0.5,1]
11
12         let fixed = (recip * 256.0).round() as u8; // Q0.8 format
13
14         writeln!(file, "{:08b}", fixed)?;
15     }
16
17     Ok(())
18 }
```

rust

### 5.2.3 Newton–Raphson Iteration

With an appropriate number of iterations, the Newton–Raphson method produces a progressively better approximation  $x_i \approx 1/f_b$ . Since all computations are performed in finite precision, each step must include explicit rounding. We model rounding using a chop operator that removes all bits after position  $\sigma$ :

$$\lfloor f \rfloor_\sigma = \lfloor f \cdot 2^\sigma \rfloor \cdot 2^{-\sigma}, \quad (73)$$

In particular,  $\lfloor f \rfloor_0$  corresponds to integer truncation.

$$\lfloor f \rfloor_0 = \lfloor f \rfloor. \quad (74)$$

#### 5.2.3.1 Finite-precision iteration

In exact arithmetic, the iteration is

$$x_{i+1} = x_i \cdot (2 - f_b \cdot x_i) \quad (75)$$

In finite precision, both multiplication and subtraction must be evaluated with truncation. This leads to

$$x_{i+1} = \left\lfloor x_i \cdot \lfloor 2 - f_b \cdot x_i \rfloor_\sigma \right\rfloor_\sigma \quad (76)$$

So each step consists of:

- computing  $z = f_b \cdot x_i$
- forming an approximation of  $2 - z$
- multiplying again with controlled truncation

#### 5.2.3.2 Simplified subtraction

Let  $z = f_b \cdot x_i$ . During convergence,  $z$  lies in  $(1, 2)$  and can be written as

$$z = \langle z[0].z[1 : s] \rangle \quad (77)$$

An exact subtraction would require forming the two’s complement of  $z$  and adding 1. Since the entire computation already operates with bounded precision, the increment step can be omitted without affecting convergence behavior.

Instead, we use the approximation

$$A_i = \langle \overline{z_i[0 : \sigma]} \rangle \quad (78)$$

The resulting error is bounded by

$$0 < 2 - z \leq \langle \overline{z[0 : \sigma]} \rangle + 2^{-\sigma} \quad (79)$$

which ensures that the simplified form remains valid throughout iteration.

### 5.2.3.3 Iteration form

The computation is summarized as

$$\begin{aligned} z_i &= f_b \cdot x_i \\ A_i &= \langle \overline{z_i[0 : \sigma]} \rangle \\ x_{i+1} &= \lfloor x_i \cdot A_i \rfloor_\sigma \end{aligned} \quad (80)$$

We define the approximation error

$$\delta_i = 1/f_b - x_i \quad (81)$$

which decreases with each iteration.

### 5.2.3.4 Quotient reconstruction

After convergence, the product

$$q' = f_a \cdot x_i \quad (82)$$

serves as an approximation of the quotient  $f_a/f_b$ . The final significand  $f_d$  is selected by comparing  $q' \cdot f_b$  with  $f_a$  under finite precision constraints to ensure correct rounding.

## 5.3 The steps of significands division

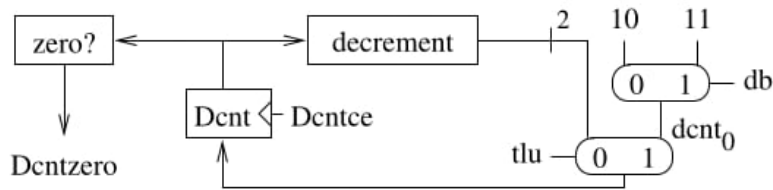


Figure 24: Counter Dcnt

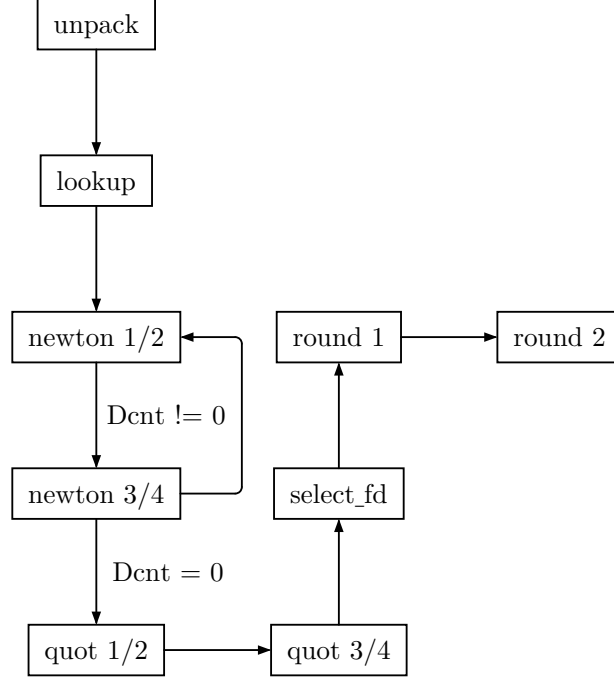


Figure 25: Control Automaton

Significand division is performed by unit **SIGFMD** under the control of the counter **Dcnt** and the **FSD**. In the original design Newton-Raphson iteration has two multiplications, each taking two cycles, giving four cycles per iteration with states **Newton 1** to **Newton 4**. Since we changed **multree** to be purely combinational multiplier inputs (**fa\_in**, **fb\_in**) can be pre-loaded in the same cycle as previous result capture, making multiplication result available on the very next posedge. This allows states **Newton\_1** - **Newton\_2** and **Newton\_3** - **Newton\_4** to be merged into single cycle states **Newton1\_2** and **Newton3\_4** respectively, reducing each iteration from four cycles to two. The counter **Dcnt** counts the number of iterations. During the table lookup, the counter is set to the number of iterations required (i.e., 2 for single and 3 for double precision), and during each iteration, **Dcnt** is counted down. After state **lookup** we have

$$x = x_0 \quad Dcnt = dcnt_0 = (db? 3 : 2). \quad (83)$$

After the  $i^{th}$  execution of state **Newton 4** we have

$$\begin{aligned} A &= A_{i-1} \\ x &= x_i \\ Dcnt &= dcnt_0 - i. \end{aligned} \quad (84)$$

The loop is left after  $i = dcnt_0$  iterations. For this  $i$ , we have after state **quotient 4**



$$E = \lfloor f_a \cdot x_i \rfloor_{p+1}$$

$$E_b = E \cdot f_b$$
(85)

After state `quotient 2` we already have

$$f_a = D_a \quad \text{and} \quad f_b = D_b.$$
(86)

| state        | RTL instruction                                                                                                                                                                                                              | control signals                                     |
|--------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------|
| unpack       | normalize FA, FB                                                                                                                                                                                                             |                                                     |
| lookup       | $x = \text{table}(f_b)$<br>$Dcnt = (db?3 : 2)$                                                                                                                                                                               | xce, tlu, fbbdoe<br>Dcntce,                         |
| Newton 1/2   | $Dcnt = Dcnt - 1$<br>$A = \text{appr}(2 - x \cdot b, 57)$                                                                                                                                                                    | Dcntce, xadoe, fbbdoe<br>Ace                        |
| Newton 3/4   | $x = \lfloor A \cdot x \rfloor_{57}$                                                                                                                                                                                         | Aadoe, xbdoe, sce, cce<br>xce                       |
| quotient 1/2 | $E = \lfloor a \cdot x \rfloor_{p+1}$<br>$D_a = f_a, D_b = f_b$                                                                                                                                                              | faadoe, xbdoe, sce, cce<br>faadoe, fbbdoe, Dce, Ece |
| quotient 3/4 | $E_b = E \cdot f_b$                                                                                                                                                                                                          | Eadoe, fbbdoe, sce, cce<br>Ebce                     |
| select fd    | $E' = E + 2^{-(p+1)},$<br>$\beta = f_a - E_b - 2^{-(p+1)} \cdot f_b$<br>$f_d = \begin{cases} E + 2^{-(p+2)} & ; \text{if } \beta < 0 \\ E' & ; \text{if } \beta = 0 \\ E' + 2^{-(p+2)} & ; \text{if } \beta > 0 \end{cases}$ |                                                     |
| Round 1/2    | round $(s_q, e_q, f_q)$                                                                                                                                                                                                      |                                                     |

Figure 26: RTL instructions of the iterative division

This table describes the control signals for SIGFMD circuit.

```

1  module div_logic(
2      input          clk,
3      input          rst_n,
4      input [52:0]   fa,
5      input [52:0]   fb,
6      input          db,
7      input          fdiv,
8      output reg [56:0] fq
9  );
10
11  logic [57:0] fa_in, fb_in;
12
13  logic [115:0] mul_out;
14
15  logic [56:0] fd_out;
```

verilog

```

16
17     logic or_out;
18     logic [7:0] rom_data;
19
20     parameter n = 60;
21
22     logic [115:0] t,s1;
23     logic [3:0] curr_state, next_state;
24     logic [3:0] Dcnt;
25     logic [57:0] x, A, Da, Db;
26     logic [54:0] E;
27     logic [115:0] Eb;
28     logic [7:0] look_up;
29
30     logic faadoe, fbbdoe;
31     logic Eadoe, Aadoe;
32     logic xadoe, xbdoe;
33
34     logic Dce, Ebce, Ece;
35     logic xce, Ace;
36     logic tlu;
37
38     rom256X8 rom_inst (.addr(fb[51:44]), .data(rom_data));
39
40     multree mlt (.a(fa_in), .b(fb_in), .out(mul_out));
41
42     ortree #(n) or_inst (.x(mul_out[59:0]), .or_out(or_out));
43
44     select_fd select_f (
45         .Da(Da),
46         .Db(Db),
47         .Eb(Eb[114:0]),
48         .E({E, 3'b0}),
49         .db(db),
50         .fd(fd_out)
51     );
52
53     localparam UNPACK      = 4'd0;
54     localparam LOOKUP      = 4'd1;
55     localparam NEWTON1_2   = 4'd2;

```

```

56  localparam NEWTON3_4  = 4'd3;
57  localparam QUOT1_2    = 4'd4;
58  localparam QUOT_3_4   = 4'd5;
59  localparam SELECT_FD  = 4'd6;
60  localparam ROUND1     = 4'd7;
61  localparam ROUND2     = 4'd8;
62
63  always @(negedge clk or negedge rst_n) begin
64      if (!rst_n)
65          curr_state = UNPACK;
66      else
67          curr_state = next_state;
68  end
69
70  always_comb begin
71      xce=0; tlu=0; Ace=0; Dce=0; Ece=0; Ebce=0;
72      faadoe=0; fbbdoe=0; Eadoe=0; Aadoe=0; xadoe=0; xbdoe=0;
73
74      case (curr_state)
75
76          UNPACK: begin
77              next_state = LOOKUP;
78          end
79
80          LOOKUP: begin
81              xce    = 1;
82              tlu    = 1;
83              xadoe  = 1;
84              fbbdoe = 1;
85              next_state = NEWTON1_2;
86          end
87
88          NEWTON1_2: begin
89              Ace    = 1;
90              Aadoe  = 1;
91              xbdoe  = 1;
92              next_state = NEWTON3_4;
93          end
94
95          NEWTON3_4: begin

```

```

96         xce = 1;
97         if (Dcnt == 3'd0) begin
98             faadoe = 1;
99             xbdoe = 1;
100             next_state = QUOT1_2;
101         end else begin
102             xadoe = 1;
103             fbbdoe = 1;
104             next_state = NEWTON1_2;
105         end
106     end
107
108     QUOT1_2: begin
109         Dce = 1;
110         Ece = 1;
111         Eadoe = 1;
112         fbbdoe = 1;
113         next_state = QUOT_3_4;
114     end
115
116     QUOT_3_4: begin
117         Ebce = 1;
118         next_state = SELECT_FD;
119     end
120
121     SELECT_FD: begin
122         next_state = ROUND1;
123     end
124
125     ROUND1: begin
126         next_state = ROUND2;
127     end
128
129     ROUND2: begin
130         // terminal state
131     end
132
133     default: begin
134         next_state = UNPACK;
135     end

```

```

136
137     endcase
138 end
139
140 always @(posedge clk or negedge rst_n) begin
141     if (!rst_n) begin
142         Dcnt     = 3'd0;
143         x         = '0; A = '0; Da = '0; Db = '0;
144         E         = '0; Eb = '0;
145         fa_in     = '0; fb_in = '0;
146         look_up   = '0; fq = '0;
147     end else begin
148
149         if (curr_state == LOOKUP)
150             Dcnt = db ? 3'd3 : 3'd2;
151         else if (curr_state == NEWTON1_2)
152             Dcnt = Dcnt - 1;
153
154         if (curr_state == LOOKUP)
155             look_up = rom_data;
156
157         if (xce) begin
158             if (tlu) begin
159                 x = {2'b01, look_up, 48'b0};
160             end else begin
161                 x = mul_out[114:57];
162             end
163         end
164
165         if (Ace) begin
166             A = ~mul_out[114:57]; // [0:57]
167         end
168
169         if (Dce) begin
170             Da = {fa, 5'b0};
171             Db = {fb, 5'b0};
172         end
173
174         if (Ece)
175             E = {mul_out[114:89], (mul_out[88:60] & {29{db}})};

```

```

176
177     if (Ebce) begin
178         Eb = mul_out[115:0];
179     end
180
181     if (faadoe) fa_in = {fa, 5'b0};
182     if (fbbdoe) fb_in = {fb, 5'b0};
183
184     if (xadoe) fa_in = x;
185     if (xbdoe) fb_in = x;
186
187     if (Eadoe) fa_in = {E, 3'b0};
188     if (Aadoe) fa_in = A;
189
190     if (curr_state == ROUND1)
191         fq = fd_out;
192
193     if (curr_state == ROUND2) begin
194         fq = fd_out;
195     end
196
197 end
198 end
199
200 endmodule

```



$$\beta = f_a - E_b - 2^{-p-1} \cdot f_b. \quad (90)$$

The sign of  $\beta$  determines the position of the exact quotient relative to the midpoint between adjacent representable candidates. Therefore, the final representative  $f_d$  is selected as

$$f_d = \begin{cases} E' + 2^{-p-2} & \beta < 0 \\ E' & \beta = 0 \\ E' + 2^{-p-2} & \beta > 0 \end{cases} \quad (91)$$

where

$$E' = E + 2^{-p-1}. \quad (92)$$

Thus, **SELECTFD** can be understood as a small correction circuit around the quotient approximation. If the residual indicates that the current approximation is below the desired representative, the circuit selects the incremented candidate. If the approximation is exactly at the midpoint, it keeps the central value. Otherwise, it selects the decremented candidate. This produces a quotient representative that is suitable for the later IEEE rounding stage.

From a hardware perspective, the circuit consists of three main parts. First, an incrementer computes  $E' = E + 2^{-p-1}$ . Second, an adder/subtractor unit evaluates the residual  $\beta$  by combining  $f_a$ ,  $E_b$ , and the shifted divisor term  $2^{-p-1} \cdot f_b$ . Third, a zero detector and sign logic distinguish the three cases  $\beta < 0$ ,  $\beta = 0$ , and  $\beta > 0$ , and drive multiplexers that select the final value of  $f_d$ .

The precision-control signal **db** selects between the single-precision and double-precision datapaths. In double precision the circuit uses the full quotient approximation range, while in single precision only the corresponding lower-precision part is used. This allows the same division datapath to support both IEEE single and double precision formats.

Therefore, the role of **SELECTFD** is not to perform the full division itself, but to perform the final mathematically justified selection of the quotient representative. This makes the following rounding stage simpler, because it receives a correctly positioned significand candidate together with the information needed for IEEE-compliant rounding.



```

1  module select_fd(
2      input  [57:0]  Da,
3      input  [57:0]  Db,
4      input  [114:0] Eb,
5      input  [57:0]  E,
6      input          db,
7
8      output [56:0]  fd
9  );
10
11  wire [115:0] a, b, c;
12  wire [116:0] t, s;
13  wire [116:0] beta;
14  wire [55:0] r;
15  wire neg, zero;
16
17  assign a = {1'b0, Da, 56'b0, 1'b1};
18  assign b = {1'b1, ~Eb[114:0]};
19  assign c = db ? {{26{1'b1}}, ~({29'b0, Db}), 3'b111}
20              : {{26{1'b1}}, ~({Db, 29'b0}), 3'b111};
21
22  parameter n = 116;
23  three2add #(n) add(
24      .a(a),
25      .b(b),
26      .c(c),
27      .c_in(1'b0),
28      .t(t),
29      .s(s)
30  );
31
32  parameter m = 117;
33  wire [117:0] sum;
34  add #(m) ad(
35      .a(t),
36      .b(s),
37      .c_in(1'b1),
38      .sum(sum)
39  );
40

```

```

41 assign beta = sum[116:0];
42 assign neg = sum[116];
43 assign zero = (beta == 117'b0);
44
45 assign r = neg ?
46     {1'b0, E[57:3]}
47     : db ?
48     (E[57:3] + 1)
49     : {E[57:32], {29{1'b1}}} + 1;
50
51 assign fd[56:30] = r[55:29];
52 assign fd[29] = db ? r[28] : ~zero;
53 assign fd[28:1] = r[27:0];
54 assign fd[0] = ~zero & db;
55
56 endmodule

```

## 5.4 Exception Flags

In the division datapath, small exception flags are generated to identify special or invalid input combinations before the normal significand division is executed. These flags indicate cases such as division by zero, operations involving NaN or infinity, and zero-result conditions, so that the datapath does not treat them as ordinary numerical divisions. Therefore, the exception logic separates IEEE special-case handling from the main Newton–Raphson based division flow and forwards the appropriate control information to the final result-selection and rounding stages.

```

1  module specmd(
2      input [52:0] nan,
3      input [3:0] fla, // [Zeroa, INFa, SNANa, NANA]
4      input [3:0] flb, // [Zerob, INFb, SNbNb, NbNb]
5      input fdiv,
6
7      output [57:0] flq
8  );
9
10 wire ZEROq;
11 wire INFq;
12 wire NANq;
13 wire INV;

```

verilog

```

14 wire DBZ;
15
16 assign DBZ = fdiv & flb[3] & ~(fla[3] | fla[2] | fla[1] | fla[0]);
17
18 assign INV = fdiv ? ((fla[3] & flb[3]) | (fla[2] & flb[2]) | (fla[1] | flb[1]))
19                 : ((fla[2] & flb[3]) | (fla[3] & flb[2]) | (fla[1] | flb[1]));
20
21 assign NANq = INV | (fla[0] | flb[0]);
22
23 assign INFq = fdiv ? ((fla[2] & ~NANq) | DBZ) :
24                 ((fla[2] | flb[2]) & ~NANq);
25
26 assign ZEROq = fdiv ? ((fla[3] | flb[2]) & ~NANq) :
27                 ((fla[3] | flb[3]) & ~NANq);
28
29 assign flq = {nan, ZEROq, INFq, NANq, INV, DBZ};
30
31 endmodule

```

## 6 Rounder

The floating-point rounder forms the final stage of the floating-point unit (FPU), where the exact result of a computation is converted into a representable floating-point number.

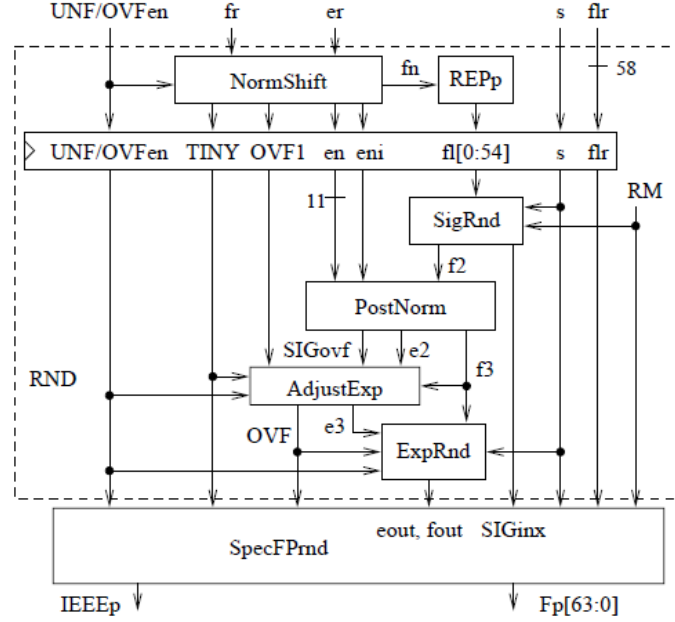


Figure 28: Schematics of the floating point rounder FPRND

Let  $x \in \mathbb{R} \setminus \{0\}$  denote the exact finite result produced by the preceding pipeline stages. Before rounding, this value may be scaled to account for overflow or underflow conditions:

$$y = \begin{cases} 2^{-\alpha} \cdot x & \text{if } OVF \wedge OVFe_n \\ 2^{\alpha} \cdot x & \text{if } UNF \wedge UNFe_n \\ x & \text{otherwise} \end{cases} \quad (93)$$

The scaled value is then decomposed into its internal representation

$$\eta(y) = (s, e, f) \quad (94)$$

The purpose of this circuit is to compute the normalized, packed output factoring  $(s, e_{out}, f_{out})$  such that  $[[s, e_{out}, f_{out}]] = r(y)$ , i.e.,

$$(s, e_{out}, f_{out}) = (s, \text{expnd}(s, \text{post}(e, \text{sigrd}(s, f)))) \quad (95)$$

In addition, the circuit generates the exception flags TINY, OVF, and SIGinx. The exponent at the output stage is represented in biased format.

The circuit accepts as inputs the overflow and underflow enable masks  $OVFe_n$  and  $UNFe_n$ , the rounding mode  $RM[1:0]$ , and the control signal  $dbr$ . It also operates on

the intermediate factoring  $(s, e_r, f_r)$ , where  $e_r$  is a 13-bit two's complement exponent and  $f_r[-1 : 55]$  includes two integer bits left of the binary point. The condition  $f_r[-1 : 0] = 00$  implies  $OVF = 0$ , ensuring that for large magnitudes of  $x$ , the significand satisfies  $f_r \in [1, 4)$ .

Finally, the **NORMSHIFT** stage provides an approximate overflow detection condition:

$$OVF1 \equiv 2^{e_r} f_r \geq 2^{e_{max}+1} \quad (96)$$

## 6.1 Normalization Shift

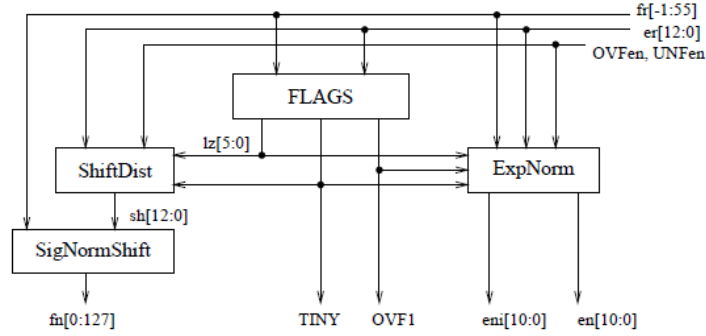


Figure 29: Circuit **NORMSHIFT** of the normalization shift

Let  $lz$  be the number of leading zeros of  $f_r[-1 : 55]$ . In general, the normalization shifter has to shift the first 1 in  $f_r$  to the left of the binary point and to compensate for this in the exponent. If the final result is a denormal number, then  $|x|$  must be represented as

$$2^{e_r} f_r = 2^{e_{min}} \cdot 2^{(e_r) - e_{min}} \cdot f_r \quad (97)$$

For a wrapped exponent we might have to add or subtract  $\alpha$  in the exponent.

## 6.2 Flags

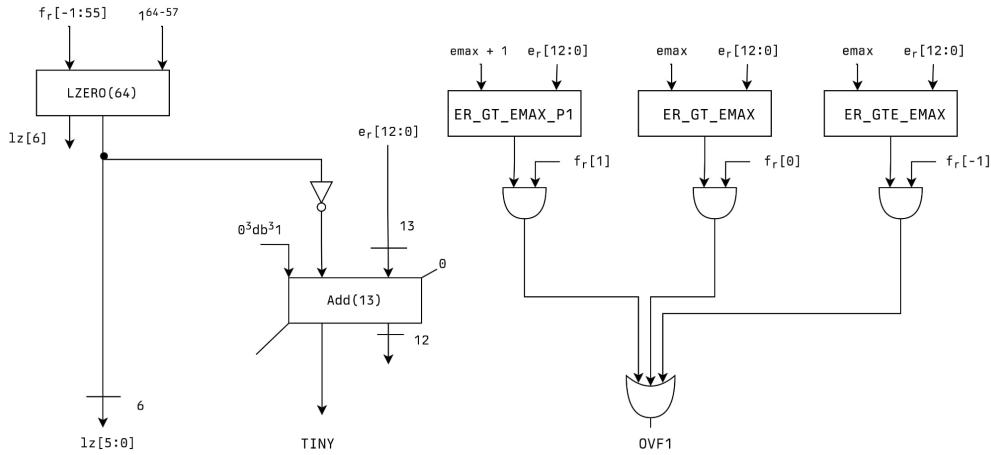


Figure 30: Circuit **FLAGS**

Circuit **FLAGS** determines the number  $lz$  of leading zeros and the flags **TINY** and **OVF1**. The computation of  $lz[5:0]$  is completely straightforward.

$$\begin{aligned} OVF1 \leftrightarrow & (f_r[-1] \wedge e_r \geq e_{max}) \vee (f_r[0] \wedge e_r > e_{max}) \\ & \vee (f_r[1] \wedge e_r > e_{max} + 1) \end{aligned} \quad (98)$$

Since  $f' = f_r/2$ , we have to consider two cases for the **TINY** flag.

$$\begin{aligned} TINY \leftrightarrow & \begin{cases} e_r + 1 < e_{min} & \text{if } f' \in [1, 2) \\ e_r + 1 - lz < e_{min} & \text{if } f' \in [0, 1) \end{cases} \\ \leftrightarrow & e_r + 1 - lz - e_{min} < 0 \end{aligned} \quad (99)$$

Thus, the **TINY** flag can be computed as the sign bit of the sum of the above 4 operands.

Recall that  $e_{min} = 1 - bias$ . Thus

$$\begin{aligned} -e_{min} + 1 &= bias - 1 + 1 = \langle 1^{n-1} \rangle \\ bias - lz &= \langle 1^{n-1} \rangle + 1 + [1\overline{lz[5:0]}] = \langle 10^{n-1} \rangle + [1^7\overline{lz[5:0]}] \\ &= \begin{cases} [0^6 1 \overline{lz[5:0]}] & \text{if } n = 8 \\ [0^3 1^4 \overline{lz[5:0]}] & \text{if } n = 11 \end{cases} \end{aligned} \quad (100)$$

```

1  module flags(
2      input  [56:0] fr,
3      input  [12:0] er,
4      input
5
6      output      TINY,
7      output      OVF1,
8      output [5:0] lz
9  );
10
11  wire [12:0] emax;
12  wire [12:0] emax_plus_1;
13
14  wire [13:0] sum;
15  wire [6:0] lz_out;
16
17  assign emax      = {3'b0, {3{db}}, {7{1'b1}}};
18  assign emax_plus_1 = {2'b0, 1'b1, ~db, {2{db}}, 7'b0};
19
20  wire er_gte_emax;
```

```

21  wire er_gt_emax;
22  wire er_gt_emax_plus_1;
23
24  leadingzero lzero(
25      .x({fr, 7'b1111111}),
26      .y(lz_out)
27  );
28
29  parameter n = 13;
30  add #(n) ad(
31      .a({3'b0, {3{db}}, 1'b1, ~lz_out[5:0]}),
32      .b(er[12:0]),
33      .c_in(1'b0),
34      .sum(sum)
35  );
36
37  assign lz = lz_out[5:0];
38  assign TINY = sum[12];
39
40  structural_gte gte (
41      .a(er),
42      .b(emax),
43      .out(er_gte_emax)
44  );
45
46  structural_gt gt_emax (
47      .a(er),
48      .b(emax),
49      .out(er_gt_emax)
50  );
51
52  structural_gt gt_emax_plus (
53      .a(er),
54      .b(emax_plus_1),
55      .out(er_gt_emax_plus_1)
56  );
57
58  assign OVFl = (fr[56] & er_gte_emax)
59               | (fr[55] & ~fr[56] & er_gt_emax)
60               | (fr[54] & ~fr[56] & ~fr[55] & er_gt_emax_plus_1);

```

```

61
62     endmodule
63
64
65     module signed_subtractor_13bit (
66         input  wire [12:0] a,
67         input  wire [12:0] b,
68         output wire [12:0] diff,
69         output wire        ovf
70     );
71
72     wire [13:0] c;
73     assign c[0] = 1'b1;
74
75     genvar i;
76     generate
77         for (i = 0; i < 13; i = i + 1) begin
78             assign diff[i] = a[i] ^ (~b[i]) ^ c[i];
79             assign c[i+1] = (a[i] & ~b[i]) | (c[i] & (a[i] ^ ~b[i]));
80         end
81     endgenerate
82
83     assign ovf = (a[12] == 1'b0 && b[12] == 1'b1 && diff[12] == 1'b1) |
84                 (a[12] == 1'b1 && b[12] == 1'b0 && diff[12] == 1'b0);
85
86     endmodule
87
88     module structural_gte (
89         input  wire [12:0] a,
90         input  wire [12:0] b,
91         output wire        out
92     );
93
94     wire [12:0] diff;
95     wire        ovf;
96
97     signed_subtractor_13bit sub (
98         .a(a),
99         .b(b),
100        .diff(diff),

```



```

101     .ovf(ovf)
102 );
103
104 assign out = ovf ? diff[12] : ~diff[12];
105
106 endmodule
107
108
109 module structural_gt (
110     input wire [12:0] a,
111     input wire [12:0] b,
112     output wire      out
113 );
114
115 wire gte_signal;
116 wire is_equal;
117
118 structural_gte gte_inst (
119     .a(a),
120     .b(b),
121     .out(gte_signal)
122 );
123
124 assign is_equal = (a == b);
125 assign out = gte_signal & ~is_equal;
126
127 endmodule

```

### 6.3 Exponent Normalization

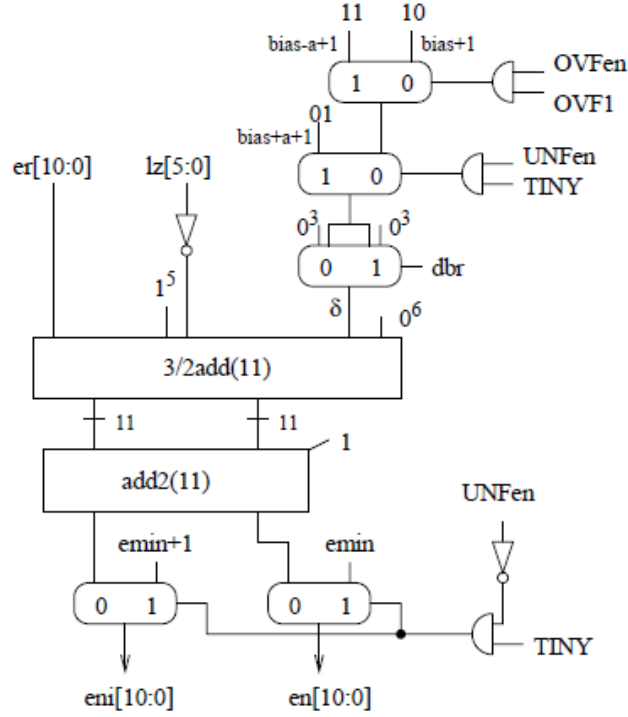


Figure 31: Circuit EXPNORM of the exponent normalization shift

The EXPNORM stage handles the special cases using two multiplexers, which directly force the output to the biased representations  $e_{\min}$  and  $e_{\min} + 1$ , encoded as  $0^{10}1$  and  $0^910$ , respectively.

For all other cases, the upper portion of the circuit computes the biased forms of  $e_n$  and  $e_n + 1$ . Equivalently, this corresponds to forming the two's complement representations of  $e_n + bias$  and  $e_n + 1 + bias$ .

In particular, let

$$\gamma = \begin{cases} -\alpha & \text{if } OVF1 \wedge OVFe_n \\ \alpha & \text{if } TINY \wedge UNFe_n \\ 0 & \text{otherwise} \end{cases} \quad (101)$$

The circuit computes the following sums  $sum$  and  $sum + 1$ :

$$\begin{aligned} sum &= e_r - lz + 1 + \gamma + bias \\ &= e_r + 1 + \left[ 1\overline{lz[5:0]} \right] + 1 + \gamma + bias \\ &= e_r + 1 + \left[ 1\overline{lz[5:0]} \right] + \delta \end{aligned} \quad (102)$$

where

$$\delta = \begin{cases} bias - \alpha + 1 & \text{if } OVF \wedge OVFe n \\ bias + \alpha + 1 & \text{if } TINY \wedge UNFe n \\ bias + 1 & \text{otherwise} \end{cases} \quad (103)$$

Recall that  $\alpha = 3 \cdot 2^{n-2} = \langle 110^{n-2} \rangle$  and  $bias = 2^{n-1} - 1 = \langle 1^{n-1} \rangle$ .

```

1  module expnorm(
2      input [10:0] er,
3      input [5:0] lz,
4      input db,
5      input OVFe n,
6      input OVFe n1,
7      input UNFe n,
8      input TINY,
9      output [10:0] eni,
10     output [10:0] en
11 );
12
13 wire [1:0] w1;
14 wire [1:0] w2;
15
16 wire [4:0] delta;
17 wire [10:0] c;
18 wire [10:0] b;
19 wire [10:0] emin;
20 wire [10:0] emin1;
21
22 wire [11:0] t, s;
23
24 assign w1 = (OVFe n & OVFe n1) ? 2'b11 : 2'b10;
25 assign w2 = (UNFe n & TINY) ? 2'b01 : w1;
26
27 assign delta = db ? {w2, 3'b0} : {3'b0, w2};
28 assign b = {5'b11111, ~lz[5:0]};
29 assign c = {delta[4:0], 6'b0};
30
31 three2add add (
32     .a(er[10:0]),
33     .b(b),
34     .c(c),
35     .c_in(1'b0),

```

verilog

```

36     .t(t),
37     .s(s)
38 );
39
40 parameter n = 11;
41 wire [11:0] sum;
42 add #(n) ad(
43     .a(t[10:0]),
44     .b(s[10:0]),
45     .c_in(1'b1),
46     .sum(sum)
47 );
48
49 assign emin = 11'b00000000001;
50 assign emin1 = 11'b00000000010;
51
52 assign en = (~UNFen & TINY) ? emin : sum[10:0];
53 assign eni = (~UNFen & TINY) ? emin1 : sum[10:0] + 1;
54
55 endmodule

```

## 6.4 Shift Distance

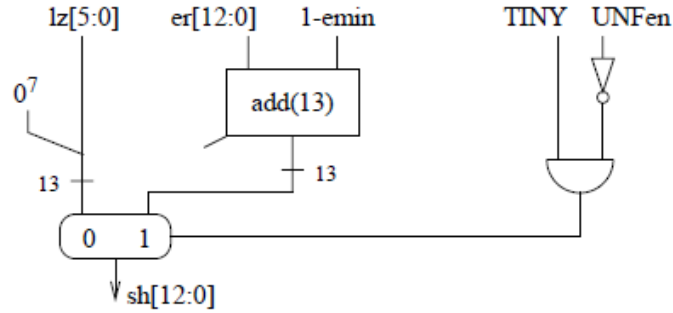


Figure 32: Circuit SHIFTDIST provides the shift distance of the normalization shift.

The SHIFTDIST unit computes the shift distance  $\sigma$ . Recall that

$$e_{\min} = 1 - bias = -2^{n-1} + 2. \quad (104)$$

Therefore,

$$1 - e_{\min} = 1 + 2^{n-1} - 2 = 2^{n-1} - 1 = \langle 1^{n-1} \rangle. \quad (105)$$

Hence, the shift output satisfies

$$sh[12:0] = \sigma. \quad (106)$$

```

1  module shiftdist(
2      input [12:0] er,
3      input [5:0] lz,
4
5      input db,
6
7      input TINY,
8      input UNFen,
9
10     output [12:0] sh
11 );
12
13 wire [12:0] emax;
14 assign emax = {3'b0, {3{db}}, {7{1'b1}}};
15
16 parameter n = 13;
17 wire [13:0] sum;
18
19 add #(n) ad(
20     .a(er[12:0]),
21     .b(emax[12:0]),
22     .c_in(1'b0),
23     .sum(sum)
24 );
25
26 assign sh = (TINY & ~UNFen) ? sum[12:0] : {7'b0, lz[5:0]};
27
28 endmodule

```

verilog

## 6.5 Significand Normalization Shift

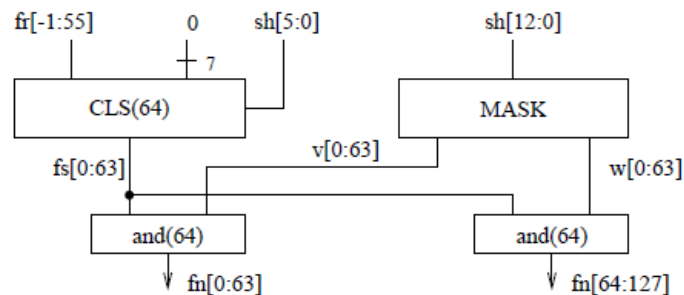


Figure 33: Circuit SIGNORMSHIFT

SIGNORMSHIFT has three parts:

1. A cyclic 64 bit left shifter whose shift distance is controlled by the 6 low order bits of  $sh$ . This takes the computation of the shift limitation in the MASK circuit off the critical path.
2. A mask circuit producing a 128 bit mask  $v[0 : 63]w[0 : 63]$
3. Let  $fs[0 : 63]$  be the output of the cyclic left shifter. Then  $f_n$  is computed by the bitwise AND of  $fs[v[]]$  and  $fs[w[]]$ .

### 6.5.1 Cyclic left shifter

For a string  $f \in \{0, 1\}^N$  and a non-negative shift distance  $d$ , we denote by  $cls(f, d)$  the result of cyclically shifting  $f$  to the left by  $d$  bits. Similarly,  $crs(f, d)$  denotes a cyclic right shift by  $d$  bits. Note that a cyclic right shift is equivalent to a cyclic left shift by the complementary distance modulo  $N$ . Hence,

$$crs(f, d) = cls(f, N - d \bmod N) = cls(f, -d \bmod N) \quad (107)$$

Let  $\sigma' = \sigma \bmod 64$ . Then, for both positive and negative shift distances, we obtain

$$[sh] = -sh[12] \cdot 2^{12} + \langle sh[11 : 0] \rangle \equiv \langle sh[5 : 0] \rangle \bmod 64 = \sigma' \quad (108)$$

```

1  module cls #(parameter N = 53)(
2      input  [5:0] m,
3      input  [N-1:0] x,
4      output [N-1:0] y
5  );
6
7  assign y = (x >> (N - m) | (x << m));
8
9  endmodule

```

verilog

### 6.5.2 Mask

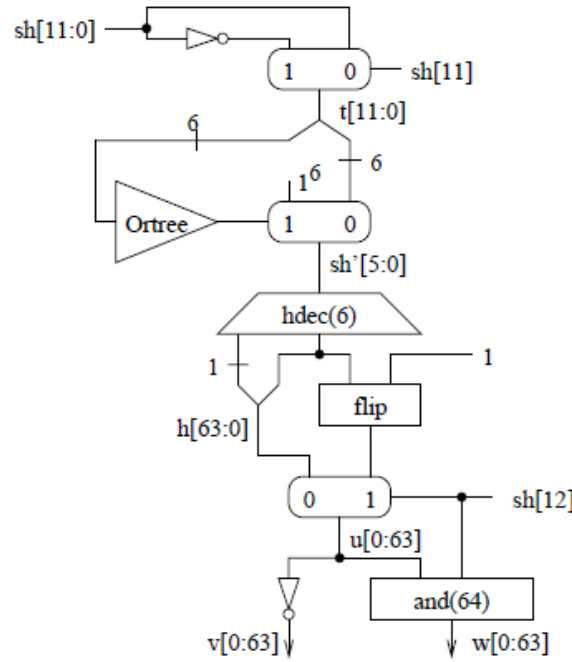


Figure 34: The MASK for the significand normalization shift

The MASK unit generates the bit masks  $v[0 : 63]$  and  $w[0 : 63]$  used during the significand normalization shift. The construction is based on the effective shift distance  $\sigma$ . The left-shift distance is bounded by  $\sigma \leq 56$ , while the circuit itself supports distances up to 63.

Accordingly, the mask generation distinguishes between positive and negative shift distances, as well as saturation when the magnitude exceeds the supported range. The resulting masks are given by

$$v[0 : 63]w[0 : 63] = \begin{cases} 1^{64-\sigma}0^{64+\sigma} & \text{if } 0 \leq \sigma \\ 0^{|\sigma|}1^{64}0^{64-|\sigma|} & \text{if } -63 \leq \sigma \leq -1 \\ 0^{64}1^{64} & \text{if } \sigma \leq -64 \end{cases} \quad (109)$$

That is, for non-negative shift distances, the mask selects the upper portion of the significand, while for negative shifts it selects the lower portion. When the shift magnitude exceeds the supported range, the masks saturate, effectively zeroing or fully enabling the corresponding segments.

```

1  module mask(
2      input  [12:0] sh,
3      output [63:0] v,
4      output [63:0] w
5  );

```

verilog

```

6
7  wire w1;
8
9  wire [11:0] t;
10 wire [5:0] shp;
11 wire [63:0] h;
12 wire [63:0] u;
13
14 assign t = sh[11] ? ~sh[11:0] : sh[11:0];
15
16 parameter n = 6;
17 ortree #(n) or_tree(
18     .x(t[11:6]),
19     .or_out(w1)
20 );
21
22
23 assign shp = w1 ? 6'b111111 : t[5:0];
24
25 HDecJ #(.N(n)) hdec(
26     .x(shp),
27     .y(h)
28 );
29
30 assign u = sh[12] ? flip_bits({h[62:0], 1'b1}) : h[63:0];
31
32 assign v = ~u;
33 assign w = u & {64{sh[12]}};
34
35
36 function automatic [63:0] flip_bits(input [63:0] in);
37     integer i;
38     reg [63:0] temp;
39 begin
40     for (i = 0; i < 64; i = i + 1)
41         temp[i] = in[63 - i];
42     flip_bits = temp;
43 end
44 endfunction
45

```



## 6.6 Selection of the representatives

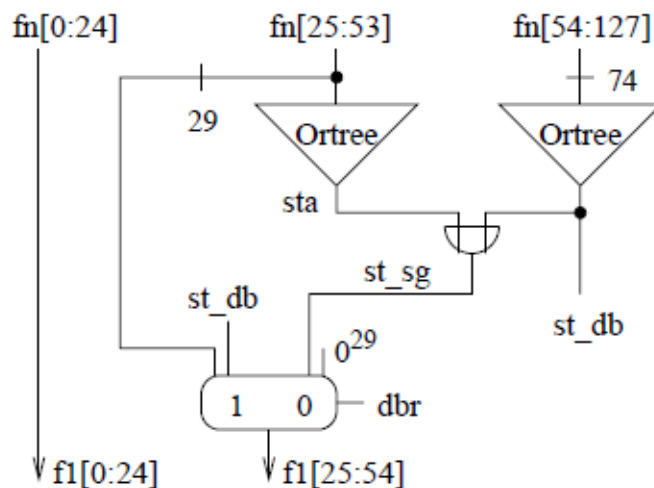


Figure 35: Circuit REPP computes the p-representative of  $f_n$ .

REPP computes a sticky bit in the following way:

$$st = \bigvee_{i \geq p+1} f_n[i] \quad (110)$$

where  $p$  depends on the precision and is either 24 or 53. The circuit selects the  $p$ -representative of  $f_n$  as

$$f_1[-1 : p + 1] = (f_n[-1 : p], st). \quad (111)$$

```

1  module rept(
2      input [127:0] fn,
3      input          db,
4
5      output [54:0] f1
6  );
7
8  parameter n1 = 74;
9  parameter n2 = 29;
10
11 wire sta, st_db;
12
13 ortree #(.n(n1)) or1(
14     .x(fn[73:0]),
15     .or out(st_db)

```

```
verilog
```

```

16 );
17
18 orte #(.n(n2)) or2(
19     .x(fn[102:74]),
20     .or_out(sta)
21 );
22
23
24
25 assign f1[54:30] = fn[127:103];
26 assign f1[29:0] = db ? {fn[102:74], st_db} : {sta | st_db, 29'b0};
27
28 endmodule

```

## 6.7 Significand Rounding

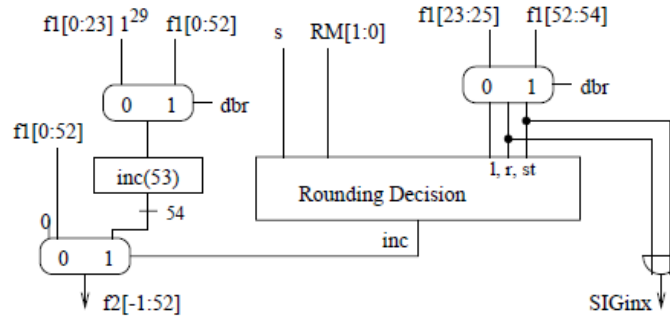


Figure 36: Circuit SIGRND

In Figure 36, the least significant bit  $l$ , round bit  $r$  and sticky bit  $st$  are selected depending on the precision and fed into circuit **ROUNDINGDECISION**. The rounded significand is exact iff the bits  $r$  and  $st$  are both zero:

$$SIGinx = r \vee st \quad (112)$$

Depending on the rounding decision,  $f_1[0 : p + 1]$  is chopped or incremented at position  $p-1$ . More formally

$$f_2 = \begin{cases} |f_1[0 : p - 1]| & \text{if } inc = 0 \\ |f_1[0 : p - 1]| + 2^{-(p-1)} & \text{if } inc = 1 \end{cases} \quad (113)$$

### 6.7.1 Rounding Decision

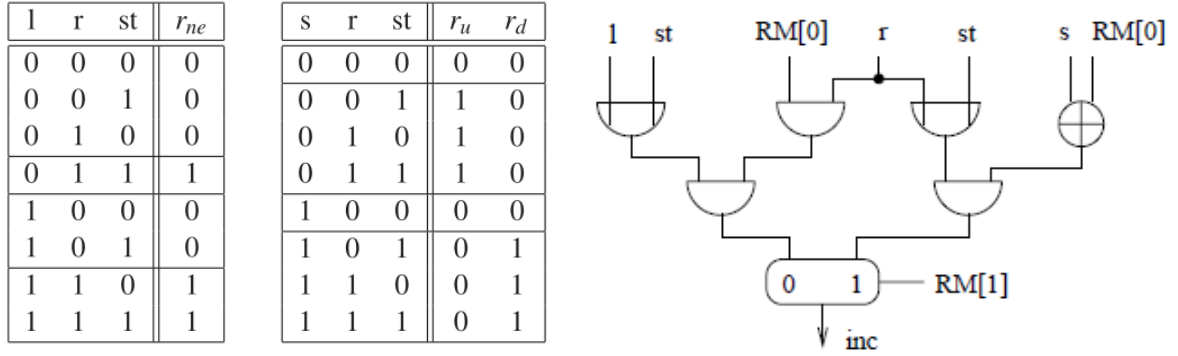


Figure 37: Rounding decision of the significand rounding

The rounding decision is determined according to Figure 37, which is constructed so that

$$f_2 = \text{sigrd}(s, f_1) \quad (114)$$

holds for every rounding mode.

In rounding mode  $r_{ne}$  (round to nearest, ties to even), the decision depends on the bits  $l$ ,  $r$ , and  $st$ , and is independent of the sign bit  $s$ . In contrast, for the directed rounding modes  $r_u$  and  $r_d$ , the decision depends on  $r$ ,  $st$ , and the sign bit  $s$ , but not on  $l$ . In mode  $r_z$ , the significand is always truncated, i.e., no increment is applied.

$$\text{inc} = \begin{cases} r \wedge (l \vee st) & \text{if } r_{ne} \\ \bar{s} \wedge (r \vee st) & \text{if } r_u \\ s \wedge (r \vee st) & \text{if } r_d \end{cases} \quad (115)$$

```

1  module roundingdecision(
2      input l,
3      input r,
4      input st,
5      input s,
6      input [1:0] RM,
7
8      output inc
9  );
10
11  assign l1 = l | st;
12  assign l2 = r & RM[0];
13  assign l3 = l1 & l2;
14
15  assign r1 = st | r;
16  assign r2 = s ^ RM[0];

```

verilog

```
17 assign r3 = r1 & r2;
18
19 assign inc = RM[1] ? r3 : l3;
20
21
22 endmodule
```

## 6.8 Post Normalization

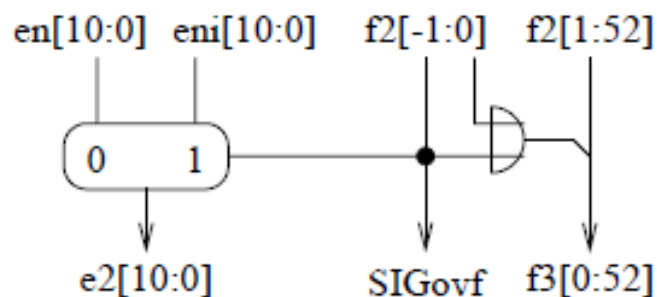


Figure 38: Post normalization circuit POSTNORM

The rounded significand  $f_2$  lies in the interval  $[1, 2]$ . If  $f_2 = 2$ , the circuit **POSTNORM** renormalizes the representation and signals a significand overflow by setting  $SIGovf = 1$ . This overflow flag is defined as

$$SIGovf = f_2[-1] \quad (116)$$

In addition, circuit **POSTNORM** computes the updated exponent–significand pair

$$(e_2, f_3) = \begin{cases} (e_n + 1, 1) & \text{if } f_2 = 2 \\ (e_n, f_2) & \text{otherwise} \end{cases} \quad (117)$$

Since the normalization shifter `NORMSHIFT` already provides  $e_n$  and  $e_{ni} = e_n + 1$ , the exponent  $e_2$  can just be selected based on the flag *SIGovf*.

$$f_3[0].f_3[1 : 52] = (f_2[1] \vee f_2[0]).f_3[0 : 52] = \begin{cases} 1.0^{52} & \text{if } f_2 = 2 \\ f_2[0 : 52] & \text{otherwise} \end{cases} \quad (118)$$

```

1  module postnorm(
2      input [10:0] en,
3      input [10:0] eni,
4      input [53:0] f2,
5
6      output sigovf,
7      output [10:0] e2,
8      output [52:0] f3
9  );
10
11  assign sigovf = f2[53];
12  assign e2 = f2[53] ? eni : en;
13  assign f3 = {f2[53] | f2[52], f2[51:0]};
14
15  endmodule

```

verilog

## 6.9 Exponent Adjustment

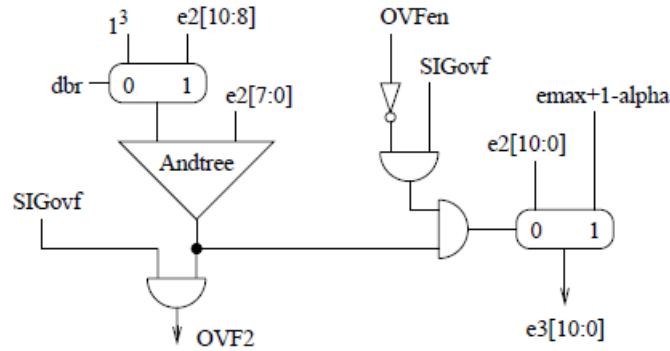


Figure 39: Circuit ADJUSTEXP

The circuit shown in Figure 39 compensates for the error introduced by the  $OVF1$  signal. The second error condition, denoted by  $OVF2$ , is detected when the exponent computation produces a value exceeding the maximum representable exponent.

$$\begin{aligned}
 e_2 &= [[e_2[10:0]]]_{bias} \\
 &= e_{max} + 1
 \end{aligned}
 \tag{119}$$

Since  $[[x]]_{bias} = \langle x \rangle - bias$  we have

$$\begin{aligned}
 e_{max} &= \langle 1^{n-1} \rangle - bias \\
 e_{max} + 1 + bias &= \langle 1^n \rangle \\
 e_{max} + 1 &= [[1^n]]_{bias}
 \end{aligned}
 \tag{120}$$

Thus, the test whether  $e_2 = e_{max} + 1$  holds is simply performed by an **AND** tree. If  $OVF2 \wedge OVFe_n$  holds, exponent  $e_2$  is replaced by the wrapped  $e_{max} + 1 - \alpha$  in biased format. Note that

$$e_{max} + 1 + bias = \langle 1^n \rangle \quad \text{and} \quad \alpha = \langle 110^{n-2} \rangle \quad (121)$$

imply

$$\begin{aligned} e_{max} + 1 - \alpha + bias &= \langle 001^{n-2} \rangle \\ e_{max} + 1 - \alpha &= [[001^{n-2}]]_{bias} \end{aligned} \quad (122)$$

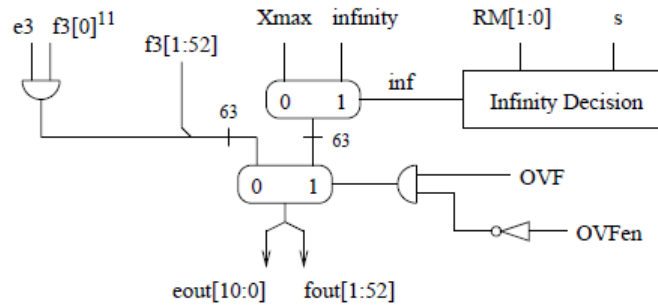
```

1  module adjexp(
2      input [10:0] e2,
3      input db,
4      input sigovf,
5      input OVF1,
6      input OVFe_n,
7
8      output [10:0] e3,
9      output OVF
10 );
11
12 wire [10:0] in;
13 wire [7:0] emax1alpha;
14 wire out;
15 wire OVF2;
16
17 assign in = db ? e2[10:0] : {3'b111, e2[7:0]};
18 assign emax1alpha = {2'b0, {3{db}}, 6'b111111};
19
20 andtree andt(
21     .x(in),
22     .and_out(out)
23 );
24
25 assign OVF2 = sigovf & out;
26 assign OVF = OVF2 | OVF1;
27
28 assign e3 = (~OVFe_n & sigovf & out) ? emax1alpha : e2[10:0];
29
30 endmodule

```

verilog

## 6.10 Exponent Rounding

Figure 40: Circuit `EXPRND`

The circuit `EXPRND` in Figure 40 computes the function `exprd` from Equation 95 and additionally converts the result into packed IEEE format. This conversion involves:

- suppressing the output bit  $f_{out}[0]$ , and
- encoding  $e_{\min}$  as  $0^n$  in the case of a denormal result.

| RM[1:0] | mode     | $s = 0$ | $s = 1$ |
|---------|----------|---------|---------|
| 00      | $r_z$    | 0       | 0       |
| 01      | $r_{ne}$ | 1       | 1       |
| 10      | $r_u$    | 1       | 0       |
| 11      | $r_d$    | 0       | 1       |

Figure 41: Table Infinity decision

In the case  $OVF \wedge \neg OVFe_n$ , the absolute values of the result is rounded to  $X_{max}$  or  $\infty$  depending on signal  $inf$ . The decision is made according to Figure 41. Circuit **Infinity Decision** implements this in a straightforward way as

$$inf = \begin{cases} RM[0] & \text{if } RM[1] = 0 \\ RM[0] \text{ XNOR } s & \text{if } RM[1] = 1 \end{cases} \quad (123)$$

Denormal significands can only occur in the case  $TINY \wedge \neg UNFen$ . In that case, we have  $e = e_{min}$  and the result is denormal iff  $f_3[0] = 0$ , i.e., if the significand  $f_3$  is denormal.

```
1  module exprnd(  
2      input s,  
3      input [10:0] e3,  
4      input [52:0] f3,  
5      input [1:0] RM,
```

```
verilog
```

```

6     input OVF,
7     input db,
8     input OVFen,
9
10    output [10:0] eout,
11    output [51:0] fout
12 );
13
14 wire inf;
15 wire [10:0] exp_xmax_inf;
16 wire [51:0] fp_xmax_inf;
17
18 assign inf = RM[1] ? ~(RM[0] ^ s) : RM[0];
19
20 assign exp_xmax_inf = inf ? {11'b1111111111} : {10'b1111111111, 1'b0};
21 assign fp_xmax_inf = inf ? 52'b0 : {{23{1'b1}}, {29{1'b1}}};
22
23
24 assign eout = (OVF & ~OVFen) ? exp_xmax_inf : e3[10:0] & {11{f3[52]}};
25 assign fout = (OVF & ~OVFen) ? fp_xmax_inf : f3[51:0];
26
27 endmodule

```

## 6.11 Circuit SPECFPRND

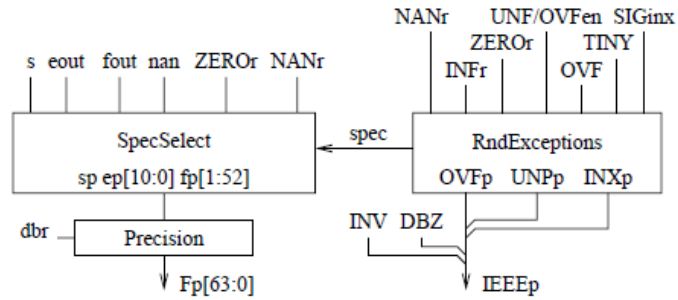


Figure 42: Circuit SPECFPRND

This circuit (Figure 42) covers the special cases and detects the IEEE floating point exceptions overflow, underflow and inexact result. In case  $a$  is a finite, non-zero number,

$$x = [[s, e_r, f_r]]_{p-\hat{e}} = a \quad (124)$$

and circuit RND already provides the packed IEEE factoring of  $x$



$$(s, e_{out}, f_{out}) = rd(s, e_r, f_r) \quad (125)$$

In case of a special operand  $a$  (zero, infinity or NaN) the flags  $flr = (ZEROr, NANr, INFr, nan, DBZ, INV)$  code the type of the operand and provide the coding of the NaN

$$nan = (s_{nan}, f_{nan}[1 : 52]) \quad (126)$$

Thus, circuit SPECSELECT computes

$$(s_p, e_p, f_p) = \begin{cases} (s_{nan}, 1^{11}, f_{nan}[1 : 52]) & \text{if } NANr = 1 \\ (s, 1^{11}, 0^{52}) & \text{if } INFr = 1 \\ (s, 0^{11}, 0^{52}) & \text{if } ZEROr = 1 \\ (s, e_{out}, f_{out}) & \text{if } spec = 0 \end{cases} \quad (127)$$

where signal  $spec$  indicates a special operand:

$$spec = NANr \vee INFr \vee ZEROr \quad (128)$$

### 6.11.1 SPECSELECT

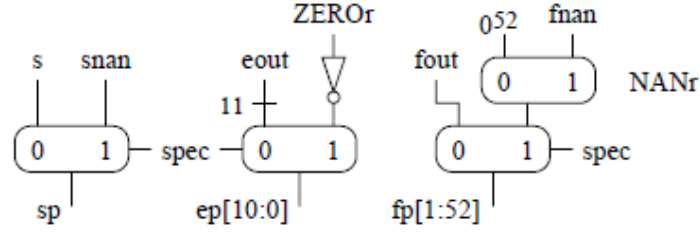


Figure 43: Circuit SPECSELECT

Depending on the flag  $dbr$ , the output factoring is either in single or double precision. The single precision result is embedded in the 64-bit word  $Fp$ . Thus,

$$Fp[63 : 0] = \begin{cases} (s_p, e_p[11 : 0], f_p[1 : 52]) & \text{if } dbr \\ (s_p, e_p[7 : 0], f_p[1 : 23], s_p, e_p[7 : 0], f_p[1 : 23]) & \text{if } \neg dbr \end{cases} \quad (129)$$

```

1  module specselect(
2      input s,
3      input [10:0] eout,
4      input [51:0] fout,
5      input [52:0] nan,
6      input ZERO,
7      input NAN,
8      input INF,
9

```

```

10     output sp,
11     output [10:0] ep,
12     output [51:0] fp
13 );
14
15 assign spec = (NAN | INF | ZERO);
16 assign sp = spec ? nan[52] : s;
17 assign ep = spec ? {11{~ZERO}} : eout;
18 assign fp = spec ? (NAN ? nan[51:0] : 52'b0) : fout;
19
20
21 endmodule

```

### 6.11.2 RNDEXCEPTIONS

In addition, circuit SPECFPRND detects the floating point exceptions  $OVF$ ,  $UNF$  and  $INX$ . These exceptions can only occur if  $a$  is a finite, non-zero number, i.e., if  $spec = 0$ . Thus,

$$\begin{aligned}
OVF_p &= \overline{spec} \wedge OVF \\
UNF_p &= \overline{spec} \wedge TINY \wedge (UNF_{en} \vee LOSS_b) \\
&= \overline{spec} \wedge TINY \wedge (UNF_{en} \vee INX) \\
INX_p &= \overline{spec} \wedge INX
\end{aligned} \tag{130}$$

Since an overflow and an underflow never occur together, signal  $INX$  can be expressed as

$$\begin{aligned}
INX &= \begin{cases} SIGinx & \text{if } (OVF \wedge OVFe_n) \vee (UNF \wedge UNFe_n) \\ SIGinx \vee OVF & \text{otherwise} \end{cases} \\
&= SIGinx \vee (OVF \wedge \overline{OVFe_n})
\end{aligned} \tag{131}$$

Circuit RNDEXCEPTIONS generates the exception flags along these equations.

```

1  module rndexceptions(
2      input NAN,
3      input INF,
4      input ZERO,
5      input OVFe_n,
6      input UNFe_n,
7      input OVF,

```

verilog

```

8     input TINY,
9     input INV,
10    input DBZ,
11    input siginx,
12
13    output reg [4:0] IEEEp
14 );
15
16 wire OVFp, UNFp, INXp, INX;
17 wire spec;
18
19 assign spec = (NAN | INF | ZERO);
20 assign OVFp = ~spec & OVF;
21 assign INX = siginx | (OVF & ~OVFen);
22
23 assign UNFp = ~spec & TINY & (UNFen | INX);
24 assign INXp = ~spec & INX;
25 assign IEEEp = {INV, DBZ, OVFp, UNFp, INXp};
26
27
28 endmodule

```

## 6.12 Operation with Special Cases

### 6.12.1 NaN signal

There are two different kinds of not a number, signaling NaN and quiet NaN. Let  $e = e[n-1:0]$  and  $f' = f[1:p-1]$ . The value represented by the floating point number  $(s, e, f')$  is a NaN if  $e = 1^n$  and  $f' \neq 0^{p-1}$ . We chose  $f[1] = 1$  for the quiet and  $f[1] = 0$  for the signaling variety of NaN.

$$[[s, e, f]] = \begin{cases} \text{quietNaN} & \text{if } e = 1^n \wedge f[1] = 1 \\ \text{signalingNaN} & \text{if } e = 1^n \wedge f[1] = 0 \wedge f' \neq 0^{p-1} \end{cases} \quad (132)$$

If an arithmetic operation involves one or two input NaNs, none of them signaling, the delivered result must be one of the input NaNs. In the specifications of the arithmetic operations, we therefore distinguish between three types of NaNs:

- qNaN denotes an arbitrary quiet NaN
- sNaN denotes an arbitrary signaling NaN
- qNaN\* indicates that the result must be one of the quiet input NaNs

### 6.12.2 Addition/Subtraction

| $a + b$   | b          |           |           |      |      |
|-----------|------------|-----------|-----------|------|------|
| a         | y          | $+\infty$ | $-\infty$ | qNaN | sNaN |
| $x$       | $r(x + y)$ | $+\infty$ | $-\infty$ |      |      |
| $+\infty$ | $+\infty$  | $+\infty$ | qNaN      |      |      |
| $-\infty$ | $-\infty$  | qNaN      | $-\infty$ |      |      |
| qNaN      | qNaN*      |           |           |      | qNaN |
| sNaN      |            |           |           |      |      |

Figure 44: Result of the addition

Recall that subtraction of two representable numbers  $a$  and  $b$  can be reduced to the addition of the two numbers  $a$  and  $c$ , where  $c$  has the factoring

$$(s_c, e_c, f_c) = (\overline{s_b}, e_b, f_b) \quad (133)$$

Therefore we just focus on the addition of two numbers. Figure 44 lists the result for the different types of operands. There are just a few cases in which floating point exceptions do or might occur:

- An *INV* exceptions occur whenever
  - one of the operands  $a, b$  is signaling NaN, or
  - when performing the operation  $+\infty - \infty$  or  $-\infty + \infty$
- The exceptions *OVF*, *UNF* and *INX* can only occur when adding two finite non-zero numbers. However, it depends on the value of the exact result, whether one of these interrupts occurs or not.

Since zero has two representations, i.e.,  $+0$  and  $-0$ , special must be paid to the sign of a zero result  $a + b$ . In case of a subtraction, the sign of a zero result depends on the rounding mode

$$x - x = -x + x = \begin{cases} +0 & \text{if } r_u, r_{ne}, r_z \\ -0 & \text{if } r_d \end{cases} \quad (134)$$

When adding two zero numbers with like signs, the sum retains the sign of the first operands, i.e., for  $x \in \{+0, -0\}$ ,

$$x + x = x - (-x) = x \quad (135)$$

### 6.12.3 Multiplication

| $a \cdot b$ | b              |      |          |      |      |
|-------------|----------------|------|----------|------|------|
| a           | y              | 0    | $\infty$ | qNaN | sNaN |
| $x$         | $r(x \cdot y)$ | 0    | $\infty$ |      |      |
| 0           | 0              | 0    | qNaN     |      |      |
| $\infty$    | $\infty$       | qNaN | $\infty$ |      |      |
| qNaN        | qNaN*          |      |          |      | qNaN |
| sNaN        |                |      |          |      |      |

Figure 45: Result of the addition

Figure 45 lists the result of the multiplication  $a \cdot b$  for the different types of operands. If the result of the multiplication is a NaN, the sign does not matter. In any other case, the sign of the result  $x = a \cdot b$  is the exclusive or of the operands' signs:

$$s_c = s_a \oplus s_b \quad (136)$$

There are just a few cases in which floating point exceptions do or might occur:

- An *INV* exception does occur whenever
  - ▶ one of the operands  $a, b$  is a signaling NaN or
  - ▶ when multiplying a zero and an infinity number, i.e.,  $0 \cdot \infty$  or  $\infty \cdot 0$
- The exceptions *OVF*, *UNF* and *INX* depend on the value of the exact result. They can only occur when both operands are finite non-zero numbers.

### 6.12.4 Division

| $a/b$    | b        |          |          |      |      |
|----------|----------|----------|----------|------|------|
| a        | y        | 0        | $\infty$ | qNaN | sNaN |
| $x$      | $r(x/y)$ | $\infty$ | 0        |      |      |
| 0        | 0        | qNaN     | 0        |      |      |
| $\infty$ | $\infty$ | $\infty$ | qNaN     |      |      |
| qNaN     | qNaN*    |          |          |      | qNaN |
| sNaN     |          |          |          |      |      |

Figure 46: Result of the addition

Figure 46 lists the result of the division  $a/b$  for the different types of operands. The sign of the result is determined as for the multiplication. This means that except for a NaN, the sign of the result  $c$  is the exclusive or of the operands' signs:  $s_c = s_a \oplus s_b$ . In the following cases, the division signals a floating point exception:

- An *INV* exception does occur whenever

- ▶ one of the operands  $a, b$  is a signaling NaN or
- ▶ when performing the operation  $\frac{0}{0}$  or  $\frac{\infty}{\infty}$ .
- A *DBZ* (division by zero) exception is signaled whenever dividing a finite non-zero number by zero.
- The exceptions *OVF*, *UNF* and *INX* depend on the value of the exact result. They can only occur when both operands are finite non-zero numbers.

## 7 Review And Result Analysis

After presenting the architecture and implementation of the floating-point unit, this chapter demonstrates its behavior through simulation. The purpose of this section is to illustrate the functional operation of each arithmetic unit at the signal level using representative test cases.

Simulation provides visibility into internal datapath signals, allowing the transformation of IEEE-754 operands to be observed across stages such as exponent handling, significand processing, normalization, and iterative refinement in the case of division.

All simulations were performed using ModelSim. Only a small set of selected test cases is presented. These cases are chosen to highlight the distinct behavioral characteristics of each unit rather than to provide exhaustive verification. Comprehensive functional testing, including large randomized test sets and corner-case validation, is addressed separately using a software-based verification framework in the following chapter.

### 7.1 Adder

The floating-point adder is demonstrated using a representative test case that highlights exponent alignment and significand shifting behavior.

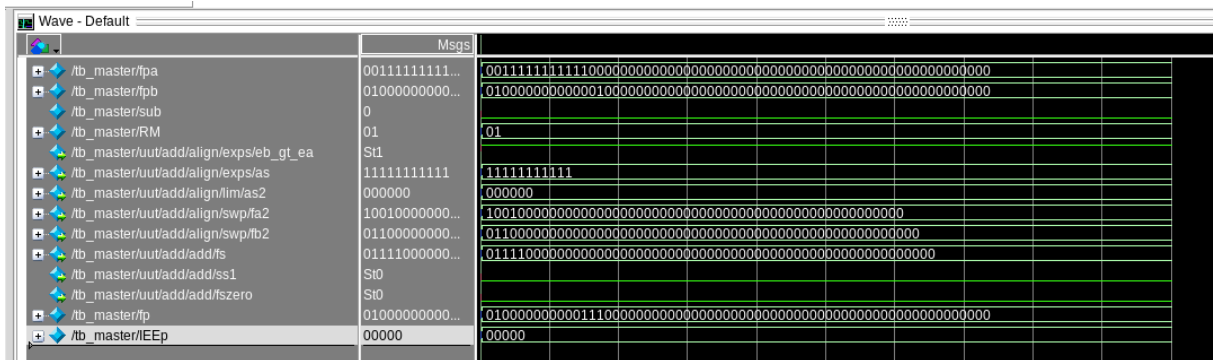


Figure 47: Adder simulation

The simulation was performed using the following inputs:

```

1 initial begin
2     // operation: 1.5 + 2.25
3     fpa = 64'b0011111111110000000000000000000000000000000000000;
4     fpb = 64'b010000000000010000000000000000000000000000000000;
5     db = 1;
6     md = 0;
7     normal = 0;
8     sub = 0;
9     fddiv = 0;
10    RM = 2'b01;
11    #20;
12
13    // expected output: 3.75 in ieee754 format
14    // 0100000000001110000000000000000000000000000000000000000
15 end

```

The operands are provided in packed IEEE-754 format and processed through the adder datapath. Since the operands have different exponents, the alignment stage shifts the smaller significand to match the larger exponent before arithmetic is performed.

The waveform highlights the behavior of the exponent comparison logic, shift selection, and sticky-bit accumulation during alignment. After alignment, the significands are combined in the addition stage, producing an intermediate result that may require renormalization.

Finally, the result is normalized and passed to the rounding stage, producing the IEEE-754 output at fp. The exception signal IEEE<sub>p</sub> remains inactive in this case.

The subtractor reuses the same datapath as the adder, with subtraction realized through sign inversion and controlled operand routing.

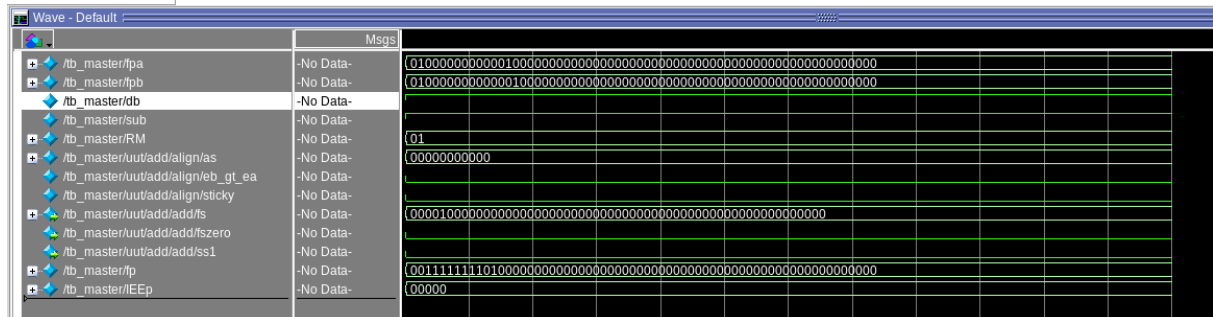


Figure 48: Subtractor simulation

The simulation was performed using the following test case:

```
1 initial begin
2     // operation: 2.5 - 2.25
3     // 2.5
4     fpa = 64'b01000000000010000000000000000000000000000000;
5     // 2.25
6     fpb = 64'b01000000000010000000000000000000000000000000;
7     db = 1; // double precision
8     md = 0; // multiply-divide = 0
9     normal = 0;
10    sub = 1; // subtraction enabled
11    fddiv = 0;
12    RM = 2'b01; // rounding mode
13    #20;
14
15    // expected output: 0.25 in ieee754 format
16    // 00111111101000000000000000000000000000000000000000000
17 end
```

The inputs are processed through the same exponent comparison and alignment logic as in the adder. However, the sub control signal inverts the effective sign of the second operand before entering the significand stage.

Since both operands share the same exponent in this example, no significant alignment shift is required. The main effect of the operation is therefore observed in the significand path.



The waveform shows that the intermediate result becomes smaller in magnitude, requiring normalization to restore IEEE-754 format. After normalization, the result is rounded and written to fp. No exception flags are asserted.

This simulation confirms that subtraction is structurally unified with addition, with differences handled primarily through control logic rather than datapath duplication.

### 7.3 Multiplier

The floating-point multiplier is demonstrated using a simple symmetric test case. The simulation verifies correct operation of partial-product generation, reduction, normalization, and rounding.

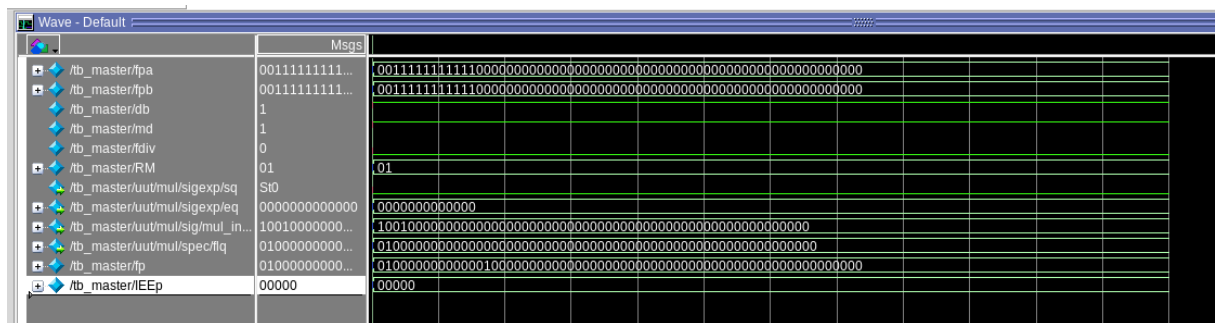


Figure 49: Multiplier simulation

The simulation applies a simple multiplication of two identical operands:

```

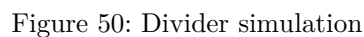
1 initial begin
2     // operation: 1.5 x 1.5
3     fpa = 64'b00111111111111000000000000000000000000000000000000;
4     fpb = 64'b00111111111111000000000000000000000000000000000000;
5
6     db = 1; // double precision
7     md = 1; // multiply-divide = 1
8     normal = 1;
9     sub = 0;
10    fdiv = 0; // division = 0
11    RM = 2'b01; // rounding mode
12    #20;
13
14    // expected output: 2.25 in ieee754 format
15    // 0100000000000010000000000000000000000000000000000000000
16 end

```

The sign is computed using an XOR operation between operand sign bits. The exponent is obtained by adding the biased exponents. The significands are multiplied using a partial-product structure followed by a reduction tree, producing an extended-precision intermediate result.

The final output is generated by the rounding stage and written to fp. The exception signal remains inactive. The result corresponds to the expected value of 2.25.

The divider simulation highlights the iterative nature of Newton–Raphson based reciprocal computation. Unlike combinational units, the division datapath is sequential and requires multiple clock cycles to converge.



```
verilog
```



- Edge cases: special IEEE-754 values such as NaN,  $\pm$ Infinity, and  $\pm 0$

## 7.6 Test Case Generation

The numerous input cases were generated by a Rust program rather than written manually one by one. For normal operands, a collection of representative floating-point values was produced using a random-number generator and then stored as fixed `f32` and `f64` pairs in the `test-data.rs` source file. Keeping the generated values fixed makes the test suite reproducible, because every run uses the same operands.

Denormal and special values were generated differently. Denormal operands were created directly from raw IEEE-754 bit patterns using `from_bits`, so that very small denormal values and the largest denormal values were included exactly as a pair.

For example:

```
1      (f32::from_bits(0x00000001), f64::from_bits(0x0000000000000001)),
```

rust

The edge-case set was built from explicit IEEE-754 values such as NaN, positive and negative infinity, zero, and the minimum and maximum finite floating-point values.

```
1      (f32::MAX, f64::MAX),
2      (-f32::MAX, -f64::MAX),
3      (f32::MIN, f64::MIN),
4      (-f32::MIN, -f64::MIN),
5      (f32::INFINITY, f64::INFINITY),
6      (-f32::INFINITY, -f64::INFINITY),
7      (f32::NAN, f64::NAN)
```

rust

The Rust generator then expanded each category into two-operand test cases by iterating over all ordered pairs of values. For example, if a category contains  $N$  values, the nested loops generate  $N \cdot N$  operand pairs. Before writing the cases to the input files, each value is decomposed into its sign, exponent, and fraction fields. These bit strings are then used by the hardware testbench and compared against the software reference result.

Each category is expanded into a deterministic test set:

- 4096 normal test cases per operation
- 64 denormal test cases per operation
- 64 edge case test cases per operation

All test cases are executed for both `f32` and `f64` implementations.

To ensure reproducibility, test data is generated programmatically. Each floating-point value is decomposed into IEEE-754 components before comparison.

```
1 pub fn decompose_f32(fp: f32) -> (u8, u8, u32) {
2     let (sign, exp, frac) = fp.decompose();
3     let s = if sign { 1 } else { 0 };
4     let e = (exp + 127) as u8;
5     (s, e, frac)
6 }
```

rust

Listing 1: IEEE-754 decomposition used for verification

The correctness check is performed by comparing the decomposed bit representation of the computed result with the reference output:

```
1
2 let result = operation(a, b);
3 let (s, e, f) = decomposer(result);
4
5 assert_eq!(
6     parsed_computed,
7     expected_output,
8     "Mismatch at test case {}",
9     i + 1
10 );
```

rust

Listing 2: Bit-level comparison of computed and reference results

## 7.7 Results

The test suite was executed for all supported operations and both floating-point formats (f32 and f64). Each operation was evaluated using the full set of normal, denormal, and edge case inputs.

In total, each operation consists of 4224 test cases per precision format. The results were determined by comparing the hardware output against the software reference model at the IEEE-754 bit level. Each test case yields a pass or fail result. In case of a mismatch, detailed diagnostic information is produced, including input operands, expected bit pattern, and computed result. Across all executed test suites, the implementation shows consistent behavior with the reference model for all test cases. All logs and detailed outputs are included in the appendix.

```

FloatingPointUnit/ieee754_test_suite on  develop  is  v0.1.0  via  v1.92.0
) cargo nextest run
  Finished `test` profile [unoptimized + debuginfo] target(s) in 0.02s

Nextest run ID b144cebc-17d7-4bb5-b753-d006c56569fd with nextest profile: default
Starting 24 tests across 18 binaries
PASS [ 0.015s] ieee754_test_suite::div_32_test test_div_denormal_f32
PASS [ 0.015s] ieee754_test_suite::mul_32_test test_mul_f32_denormal
PASS [ 0.015s] ieee754_test_suite::mul_32_edge_test test_mul_edge_f32
PASS [ 0.015s] ieee754_test_suite::add_32_test test_add_f32_denormal
PASS [ 0.016s] ieee754_test_suite::add_64_test test_add_f64_denormal
PASS [ 0.018s] ieee754_test_suite::div_64_test test_div_denormal_f64
PASS [ 0.022s] ieee754_test_suite::add_32_edge_test test_add_edge_f32
PASS [ 0.021s] ieee754_test_suite::mul_64_edge_test test_mul_edge_f64
PASS [ 0.023s] ieee754_test_suite::add_64_edge_test test_add_edge_f64
PASS [ 0.024s] ieee754_test_suite::div_32_edge_test test_div_edge_f32
PASS [ 0.024s] ieee754_test_suite::div_64_edge_test test_div_edge_f64
PASS [ 0.016s] ieee754_test_suite::mul_64_test test_mul_f64_denormal
PASS [ 0.016s] ieee754_test_suite::sub_32_edge_test test_sub_edge_f32
PASS [ 0.016s] ieee754_test_suite::sub_32_test test_sub_f32_denormal
PASS [ 0.013s] ieee754_test_suite::sub_64_edge_test test_add_edge_f64
PASS [ 0.015s] ieee754_test_suite::sub_64_test test_sub_f64_denormal
PASS [ 0.047s] ieee754_test_suite::add_32_test test_add_f32
PASS [ 0.049s] ieee754_test_suite::mul_32_test test_mul_f32
PASS [ 0.049s] ieee754_test_suite::div_64_test test_div_f64
PASS [ 0.050s] ieee754_test_suite::add_64_test test_add_f64
PASS [ 0.052s] ieee754_test_suite::div_32_test test_div_f32
PASS [ 0.037s] ieee754_test_suite::mul_64_test test_mul_f64
PASS [ 0.037s] ieee754_test_suite::sub_32_test test_sub_f32
PASS [ 0.028s] ieee754_test_suite::sub_64_test test_sub_f64

Summary [ 0.057s] 24 tests run: 24 passed, 0 skipped

```

Figure 51: Execution summary of the full floating-point test suite showing 24 executed tests with 100% pass rate across all operations and formats

The full implementation is available in the project repository [2].

## 8 Conclusion

In this thesis, the floating-point processing system from Mueller and Paul [1] was implemented in SystemVerilog, following the IEEE 754 standard. The implementation covers the core arithmetic operations and handles both normal and exceptional cases, including overflow, underflow, and denormal numbers. The efficient circuit designs for addition and multiplication were taken from [3] (Figure 17), while the divider was kept close to the original from [1] with only minor modifications such as changing tri state buses to mux trees and changing control automaton for iterations.

The work demonstrated that the theoretical designs presented in these references can be successfully realized in SystemVerilog with relatively minor adaptations. The modular structure of both sources made the implementation straightforward, and leaves room for potential extensions such as additional operations like logarithm or square root.

## Bibliography

- [1] S. M. Mueller and W. J. Paul, Computer Architecture: Complexity and Correctness. Springer, 2000.
- [2] K. Mate and K. Zura, “FloatingPointUnit.” [Online]. Available: <https://github.com/Ka10kenHQ/FloatingPointUnit>
- [3] W. J. Paul, C. Baumann, P. Lutsyk, and S. Schmaltz, System Architecture: An Ordinary Engineering Discipline. Springer, 2016.